

A Cubic Barrier with Elasticity-Inclusive Dynamic Stiffness

RYOICHI ANDO, ZOZO, Japan

CCS Concepts: • **Computing methodologies** → **Physical simulation**.

Additional Key Words and Phrases: collision, contact

ACM Reference Format:

Ryoichi Ando. 2024. A Cubic Barrier with Elasticity-Inclusive Dynamic Stiffness. *ACM Trans. Graph.* 43, 6, Article 224 (December 2024), 14 pages. <https://doi.org/10.1145/3687908>

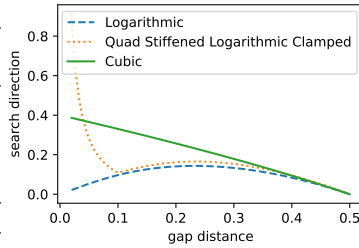
A SEARCH DIRECTION LOCKING

In this section we illustrate the issue of search direction locking through a one-dimensional example. Let $\psi_{\text{hook}}(g) = \frac{1}{2}g^2$ be a Hooke's energy and $\psi_{\text{ln}} = -\kappa(g-1)^2 \ln(g)$ be a logarithmic barrier, and we wish to find the minimum of $\psi_{\text{total}} = \psi_{\text{hook}} + \psi_{\text{ln}}$. When $\kappa = 1/(1 + \ln(4))$, the minimum is found at $g = 1/2$. A Newton's step becomes

$$g_{k+1} = g_k + d(g_k), \text{ where } d(g) = -\left(\frac{\partial^2 \psi_{\text{total}}}{\partial g^2}\right)^{-1} \frac{\partial \psi_{\text{total}}}{\partial g}, \quad (1)$$

and g_k is k -th step of g . $d(g)$ is the search direction.

Inset on the right visualizes $d(g)$. $d(g)$ is expected to diminish to zero only at $g = 1/2$, but the logarithmic barrier $d(g)$ vanishes to zero at gap zero. Huang et al. [2024] discussed the same issue and proposed a variant of logarithmic barriers, suggesting capping the high curvature at a threshold (e.g., $g = \hat{g}/10$). However, $d(g)$ now overshoots.



B NUMERICAL CONVERGENCE OF BARRIER ENERGIES

In §A we hinted that logarithmic barriers [Li et al. 2020] and their variance [Huang et al. 2024] induce convergence issues near gap zero, but it is still unclear how these issues appear in numerical settings. In this section, we provide comprehensive Python code (Listing 1) with a minimal setting and discuss the numerical behaviors of these two barriers. Our setting is as follows: we define the total energy potential as

$$\psi_{\text{total}}(y) = \frac{1}{2}y^2 + \psi_{\text{barrier}}(y), \quad (2)$$

Author's address: Ryoichi Ando, ryoichi.ando@zozo.com, ZOZO, Chiba, Japan.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM. 0730-0301/2024/12-ART224 \$15.00 <https://doi.org/10.1145/3687908>

where $\psi_{\text{barrier}}(y)$ denotes a barrier function that yields infinity at $x = 0$. More specifically, they defined as

$$\psi_{\text{log}}(y) = -k_{\text{log}}(y-1)^2 \ln(y), \quad (3)$$

$$\psi_{\text{qlog}}(y) = k_{\text{qlog}}(y^2-1)^2 \ln^2(y^2), \quad (4)$$

where k_{log} and k_{qlog} denote the stiffness with respect logarithmic and quadratic stiffened logarithmic energies, respectively. The first term in (2) can be seen as a momentum potential energy that pulls a particle toward the origin. In our example, we specifically set the stiffness to

$$k_{\text{log}} = \frac{1}{1 + \ln(4)}, \quad k_{\text{qlog}} = \frac{1}{3(\ln(4) + 3) \ln(4)} \quad (5)$$

so that the stationary solution is found at $y = 1/2$ for both energies. We modify curvature of (4) to

$$\frac{\partial^2 \psi_{\text{qlog}^*}(y)}{\partial y^2} = \min\left(\lambda_{\text{thr}}, \frac{\partial^2 \psi_{\text{qlog}}(y)}{\partial y^2} + \frac{1}{2y} \frac{\partial \psi_{\text{qlog}}(y)}{\partial y}\right), \quad (6)$$

where λ_{thr} is a curvature evaluated at $y = 0.1$ to prevent division by a significantly large number and avoid search-direction locking; that is

$$\lambda_{\text{thr}} = 19602/25 + 2277 \ln(10)/5 - 586 \ln(10)^2/25. \quad (7)$$

Using the above symbolic expressions, we can employ the Newton's method to find a stationary point given an initial guess y_0 .

Figure 1 visualizes the convergence transition and the line search attempts for each energy, starting with the initial guesses of 10^{-1} , 10^{-2} , and 10^{-3} , respectively. The logarithmic energy smoothly converges to the desired solution, but due to the large curvature near zero, the updates are subtle for the first several iterations, resulting in slower convergence. For the quadratic stiffened logarithmic barrier filtered, the iteration count may be improved, but the search direction points beyond the stationary point, necessitating multiple line searches that globally shrink the search direction.

In our case, on the other hand, stiffness dynamically changes by Newton's steps, which induce fluctuation (Figure 2). Nevertheless, our method still updates the solution reasonably close to $y = 1/2$ even in the first iteration. This implies that our inexact Newton's method can be more reliably used for collision resolution than logarithmic barriers within a limited budget of Newton's steps. Note that in practical contact scenarios the update is shrunk by the line search, so the fluctuation is less.

C EARLY-TERMINATED NEWTON'S METHOD

Our solution of motion is described by the stationary-action principle, and accordingly, Newton's method requires an action comprising elastic, momentum, and collision energy to be extremized in the admissible space. Strictly speaking, failure to comply with this rule manifests as artifacts such as ghost forces, excessive dissipation, and a violation of conservative laws. Nonetheless, it is also known that a single step of Newton's method (alternatively, implicitly linearized

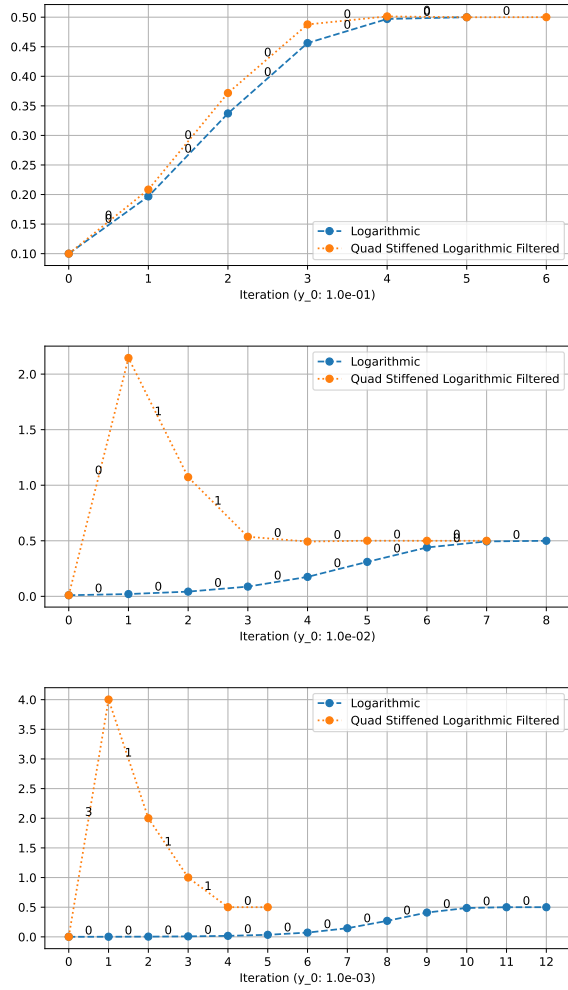


Fig. 1. Convergence to $y = 1/2$ for each energy with respect to Newton's iterations. From top to bottom: the initial guess is given by $y_0 = 0.1, 0.01, 0.001$. Numbers between points indicate the line search attempts that actually halved the search direction.

integration) can be satisfactory for visual applications [Baraff and Witkin 1998; Wang et al. 2023; Wolf et al. 2021].

Unfortunately, attempting it naively with a customized Newton-type solver presented in the IPC [Li et al. 2020] results in a dragged trajectory of motion. To see why, let us consider a single particle in ballistic motion. In this case, the action of motion is quadratic, so a single step is sufficient to reach an exact stationary point. However, when the search direction is shrunk by a line search triggered elsewhere, such as $\alpha = 0.5$ (mostly due to collision), the velocity is erroneously halved. In this specific situation, we can interpret that the time step evolved is also halved, and then the velocity is correctly preserved.

For ballistic particles, this simple strategy seems reasonable, but it is not clear if this also works for general cases. For small steps where

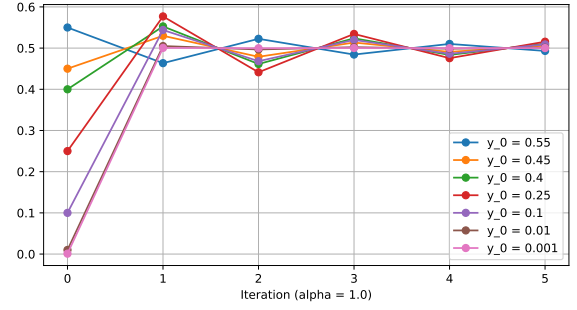


Fig. 2. Convergence to $y = 1/2$ of our cubic energy with a dynamic squared-inverse-distance stiffness $k_{\text{cube}} = 1/4y^2$, starting from various initial guesses.

ballistic motion dominates, we assess that the above treatment still provides a good guess. In fact, when particles are in collision, they do not necessarily need to stay at the exact stationary point.

An example is illustrated in Figure 3. In this setting, two particles A and B are assigned a constant velocity. Here, let x_A and x_B be two initial positions associated with the particles at time $t = 0$, and z_A and z_B be target positions resulting from inertia at time $t = \Delta t$. Since there are no static obstacles between x_B and z_B , particle B should land on z_B . On one hand, there is a static obstacle in the trajectory of particle A , and so the actual path must not cross it. In this specific situation, the ideal trajectory of A becomes unclear.

For example, when we run the full Newton's steps until a stationary state is reached, as illustrated, particle A lands exactly on z_A . We consider this to be non-physical, as this can be seen as a tunneling artifact. This issue may not be crucial as such thin objects usually do not exist.

A more pronounced issue inherent in this example is the number of Newton's steps required to reach this state. As evident from the illustration, during the optimization, particle A repeatedly bounces once it collides with the surface for the first time, landing on the surface again and again until z_A is directly visible from the current optimization position. This inefficiency aligns with the number of Newton's step counts, each entailing expensive contact assembly, energy potential derivative evaluation, and linear system solves [Wang et al. 2023]. In contrast,

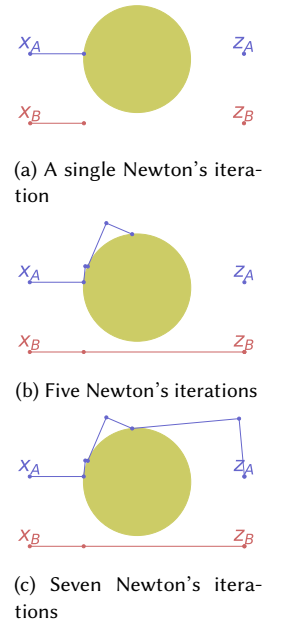


Fig. 3. Trajectory of a particle in the Newton's steps.

our early terminated Newton's method with a line-search reduced step size does not encounter the same issue.

Note that in the above, we assume a single-step terminated Newton's method; which can be problematic even in the absence of contact. For example, Wu et al [2022] suggests three newton steps for nonlinear deformables. Therefore, it must be generalized to handle multiple steps.

To begin with, let us consider a scenario where we iterate twice and track the trajectories of a ballistic particle B in motion. In the first step, assuming that its update is hindered by a line search with $\alpha = 0.5$, the particle position is updated to $(\mathbf{x}_B + \mathbf{z}_B)/2$. In the second step, assuming once again that its update is hindered by $\alpha = 0.5$, the new particle position is updated to $3(\mathbf{x}_B + \mathbf{z}_B)/4$, not exactly $\mathbf{z}_B$. This idea can be generalized as follows: if the update is truncated by α , the accumulated time advances by $\alpha(1 - \beta)$, where β denotes the normalized accumulated time ($0 \leq \beta \leq 1$) to be multiplied by Δt and returned. Finally, we arrive at our generalized inexact Newton's method.

Obviously, our inexact Newton's method does not arrive at exact stationary states, and in theory this can lead to ghost crawling motion as a side effect, especially when objects of interest are in an equilibrium state.

In many cases, this is not a concern, as momentum energy changes once we leave the Newton's loop; therefore, such inertia-pulling force is released in the next step. The remaining shape-dependent energies, such as elastic, bending and contact terms, may remain unsettled, and they could in theory induce ghost forces for some extended period of time. We have examined this carefully and conclude that this concern is negligible or tolerable.

Our temporal integrator should not be confused with the two-step approaches [Harmon et al. 2008; Narain et al. 2012; Tang et al. 2018; Wang et al. 2023; Wu et al. 2022]. They first solve for the goal position $\mathbf{x}_{\text{goal}}$ using a dynamic solver and try best to bring variables as close as possible to $\mathbf{x}_{\text{goal}}$ within the admissible region. When solved with this approach, several Newton's steps are more important since the dynamic solver advances variables to the next time step without considering possible collisions. In contrast, our method is based on the interior point method, which triggers re-linearization whenever we detect collision.

D CACHED CONTACT MATRIX ASSEMBLY

When dozens of millions of contacts are involved, matrix assembly on the GPU can be both memory intensive and time consuming.

Matrix-free methods are best memory-efficient when element-wise local matrices are constructed from scratch whenever requested. When many iterations are needed, doing so is expensive because it involves proximity query, degraded cache coherency, analytical Hessian construction, and possibly atomic additions at every CG iteration. One may store intermediate local matrices for each element such as per triangle or contact, but that would be more memory-consuming than the full matrix assembly due to matrix entry duplicates. This motivates us to choose the full matrix assembly over matrix-free methods, which also simplifies the evaluation of elasticity-inclusive stiffness.

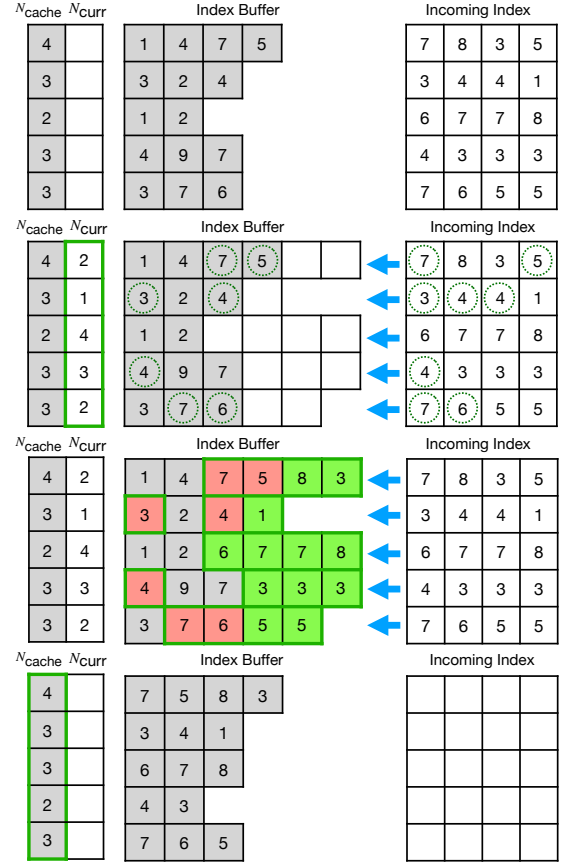


Fig. 4. Cached Contact Matrix Assembly Overview. From top to bottom: A buffer filled with cached contacts. First, we perform a dry pass that counts incoming entries not present in the cache. The counts are then used to allocate device memory. Second, we run a fill-in pass where actual entries are either merged (red cells) into the cache or appended (green cells). Finally, we deduplicate entries, also eliminating untouched caches. The compressed buffer is carried to the next step as a new cache.

Our sparse matrix assembly is based on the work of Contact-Aware Matrix Assembly (CAMA) [Tang et al. 2016], which uses a sparse 3×3 block row compressed format on the device memory. To clarify the additions we made to CAMA, let us start by providing an overview of their contact matrix assembly, followed by our extension. First, we conduct a dry pass over contact elements, counting all the possible non-zero fills for each row. These numbers are then prefix-summed and used to allocate device memory, with starting indices recorded for each row. In the second pass, we once again query for contact elements and fill the associated matrix elements, allowing duplicates. Finally, those duplicates are merged for the actual use in matrix-vector product. A pictorial workflow can be found in Figure 5 in the paper [Tang et al. 2016]. This parallel matrix assembly is sufficient for mildly contact-rich examples. However, when the count of contacts exceeds a certain upper bound (e.g., dozens of millions), the device memory allocation fails. This is primarily due to the numbers of duplicates of the matrix entries before

Algorithm 1 Cached Contact Matrix Assembly

```

1: function ASSEMBLE CONTACT MATRIX
2:   for all element-wise contact Hessian  $H_k$  in parallel do
3:     for all  $(i, j)$  : index of  $3 \times 3$  sub-matrix in  $H_k$  do
4:       ATOMIC-INCREMENT( $i, j$ )
5:     end for
6:   end for
7:   Prefix sum and allocate device memory
8:   Clear values associated with indices in cache
9:   for all element-wise contact Hessian  $H_k$  in parallel do
10:    for all  $(i, j)$  and  $3 \times 3$  sub-matrix  $Y_{\text{val}}$  in  $H_k$  do
11:      ATOMIC-ADD( $i, j, Y_{\text{val}}$ )
12:    end for
13:  end for
14:  for all row  $i$  in parallel do
15:    Merge duplicates and remove untouched cache
16:  end for
17:  Copy current indices as a new cache
18: end function
19: function ATOMIC INCREMENT( $i, j$ )
20:   if entry  $j$  is not found in  $i$ -th row's cache then
21:     Atomically increment  $i$ -th row's non-zero count
22:   end if
23: end function
24: function ATOMIC ADD( $i, j, Y_{\text{val}}$ )
25:   if entry  $j$  is found in  $i$ -th row's cache then
26:     Atomically merge  $Y_{\text{val}}$  to  $i$ -th row's cached entry  $j$ 
27:   else
28:     Append  $(j, Y_{\text{val}})$  to  $i$ -th row's list (partially atomic)
29:   end if
30: end function

```

being compressed, which can, in the worst case, consume an order of magnitude more memory than actually needed. This limitation severely hinders the scalability of our method, and such, it must be addressed.

To this end, we propose an extension to the original CAMA as illustrated in Figure 4. Our idea is to recognize that many contacts persist for an extended period of steps, and thus we can keep a record of such contacts from the previous step. Note that such an assumption is often utilized in collision processing [Cirio et al. 2014, 2017; Kaldor et al. 2010; Selle et al. 2009], but to the best of our knowledge, it has not yet been applied to CAMA.

Our extension works by reserving extra matrix-element space at the beginning of each row, representing a list of indices in the previous step, which we refer to as cached entries (grayed cells in the index buffer).

First, as before, we perform a dry run on the incoming contacts, counting the number of possible matrix entries except that if we find the same contact in the cache (dot circles), we skip counting this entry. The sum of the above and the count of cached entries on each row is similarly prefix summed and used to reserve device memory. Notice that at this stage, memory allocation is significantly reduced by not counting cached entries.

Next, we query the active contacts again and add the matrix entries to the row. During this process, if the entry is found in the cache, we merge it with the associated value instead of appending it to the row. Finally, we merge duplicates and clean up. In doing so, when the cached contacts are not touched, their associated values remain zero, and we drop them. This new list of contacts, after being merged and with the removal process applied, is then carried over to the next step as the new cache.

It should be noted that our extension introduces some extra cost to the original CAMA, both during the counting and fill-in stages. This is due to the need to scan the caches and decide if an entry should be counted as merge-able or appended. This indicates that in highly contact-intensive situations, our matrix assembly can be the most or second most expensive component in the whole pipeline. If practitioners find the extra cost too overwhelming and still have some space for memory allocation, they can skip caching some entries, for example, only caching entries with odd-numbered indices. This allows practitioners to control the balance between memory consumption and the impact of the extra cost associated with the cached entries. In a production environment, this balance may be dynamically determined based on the available device memory and contact complexity.

Regarding the added memory consumption due to the cache, we find it to be acceptable. Without our extension, the total required device memory amounts to $10N_{\text{max}}$, where N_{max} denotes a pre-defined large number, and our extension adds another N_{max} to this total. Interested readers are referred to the another supplementary material for this reasoning. Note that our method is identical to the original CAMA if no caching is applied. A pseudo code of our cached contact matrix assembly is laid in Algorithm 1.

We note that it is possible to achieve further compact device memory allocation by running an additional untouched-cache removal pass before the actual device memory allocation. However, this would also introduce the extra cost of re-ordering and increase implementation complexity. Hence, we choose to remove and deduplicate all at once in the final pass.

D.1 Dynamic Matrix Transpose

Since our global matrix is symmetric, upper triangle entries are only needed. Lower triangle entries are also built on the GPU but actual values are pointers to the corresponding upper triangle entries. Dynamic transpose is two orders of magnitude faster than explicit fill-ins and enables further reduction in the required device memory. For clarity, a pseudo code is laid in Algorithm 2.

E EIGENANALYSIS OF ISOTROPIC SHELL DISTORTIONS

In this section, we first derive a set of eigenvalues and their corresponding eigenmatrices for isotropic energies expressed in terms of singular values for shells. Next, we use these results to derive our eigenanalysis for strain-limiting energies, which enables a concise explicit expression. To begin with, let us consider the first derivative of an elastic potential $\Psi_{2D}(\sigma_1, \sigma_2)$, which is a function of singular values, with respect to mesh vertices $\mathbf{x}$

$$\frac{\partial \Psi_{2D}}{\partial \mathbf{x}} = \left(\frac{\partial \Psi_{2D}}{\partial \sigma} \right) : \left(\frac{\partial \sigma}{\partial \mathbf{x}} \right). \quad (8)$$

Algorithm 2 Dynamic Transpose on the GPU

```

1: function DYNAMIC_TRANSPOSE( $H$ )      ▶  $H$  is upper-triangle
2:    $C \leftarrow$  array of zero  $n$  (vertex count) integers
3:   for all block-wise  $3 \times 3$  matrix  $H_{ij}$  in parallel do
4:     Atomically increment  $C_j$  by 1
5:   end for
6:   Prefix sum  $C$  and allocate one-dimensional array  $A$ 
7:    $I \leftarrow$  array of zero  $n$  integers.
8:   for all block-wise  $3 \times 3$  matrix  $H_{ij}$  in parallel do
9:      $k \leftarrow C_j + \text{ATOMIC-FETCH-ADD}(I_j, 1)$ 
10:     $A_k \leftarrow$  pointer to  $H_{ij}$       ▶ For actual use transpose it
11:  end for
12:  return  $A, C$                         ▶ CSR format data
13:  ▶  $j$ -th row starts from  $C_j$ , none-zero count is  $C_{j+1} - C_j$ 
14: end function

```

Notice that (8) involves the derivative of singular values, which can be expanded as follows

$$\frac{\partial \sigma_i}{\partial x_j} = \mathbf{u}_i \cdot \left(\frac{\partial \mathbf{F}_{m \times n}}{\partial x_j} \mathbf{v}_i \right), \quad (9)$$

where $\mathbf{F}_{m \times n}$ is a deformation gradient matrix of size $m \times n$ ($m \geq n$). $\mathbf{u}_i \in \mathbb{R}^m$ and $\mathbf{v}_i \in \mathbb{R}^n$ denote two vectors corresponding to the i -th column of $\mathbf{U}_{m \times n}$ and $\mathbf{V}_{n \times n}$ matrices factored via the (inversion-aware) singular value decomposition (SVD) with respect to $\mathbf{F}_{m \times n}$. In the following, $\Sigma_{n \times n}$ denotes a diagonal singular value matrix given by $\Sigma_{ii} = \sigma_i$ and zero elsewhere. σ_j denotes a singular value of $\mathbf{F}_{m \times n}$. Next, let us focus on the second derivative of Ψ_{2D} . This is expanded as follows

$$\frac{\partial^2 \Psi_{2D}}{\partial \mathbf{x}^2} = \left(\frac{\partial \boldsymbol{\sigma}}{\partial \mathbf{x}} \right)^T \left(\frac{\partial^2 \Psi_{2D}}{\partial \boldsymbol{\sigma}^2} \right) \left(\frac{\partial \boldsymbol{\sigma}}{\partial \mathbf{x}} \right) + \left(\frac{\partial \Psi_{2D}}{\partial \boldsymbol{\sigma}} \right) : \left(\frac{\partial^2 \boldsymbol{\sigma}}{\partial \mathbf{x}^2} \right). \quad (10)$$

Using identities provided by Townsend [2016], the second derivative of $\boldsymbol{\sigma}$ with respect to $\mathbf{x}$ is given by

$$\frac{\partial^2 \sigma_k}{\partial x_i \partial x_j} = \frac{\partial \mathbf{u}_k}{\partial x_j} \cdot \frac{\partial \mathbf{F}_{m \times n}}{\partial x_i} \mathbf{v}_k + \mathbf{u}_k \cdot \frac{\partial^2 \mathbf{F}_{m \times n}}{\partial x_i \partial x_j} \mathbf{v}_k + \mathbf{u}_k \cdot \frac{\partial \mathbf{F}_{m \times n}}{\partial x_i} \frac{\partial \mathbf{v}_k}{\partial x_j}, \quad (11)$$

and the derivatives of $\mathbf{u}$ and $\mathbf{v}$ are given as

$$\frac{\partial \mathbf{u}_k}{\partial x_j} = \mathbf{U}_{m \times n} \left(C_{n \times n} \circ \left(W_{n \times n} \Sigma_{n \times n} + \Sigma_{n \times n} W_{n \times n}^T \right) \right) \mathbf{e}_k + \left(I_{m \times m} - \mathbf{U}_{m \times n} \mathbf{U}_{m \times n}^T \right) \frac{\partial \mathbf{F}_{m \times n}}{\partial x_j} \mathbf{V}_{n \times n} \Sigma_{n \times n}^{-1} \mathbf{e}_k, \quad (12)$$

$$\frac{\partial \mathbf{v}_k}{\partial x_j} = \mathbf{V}_{n \times n} \left(C_{n \times n} \circ \left(\Sigma_{n \times n} W_{n \times n} + W_{n \times n}^T \Sigma_{n \times n} \right) \right) \mathbf{e}_k, \quad (13)$$

$$W_{n \times n} = \mathbf{U}_{m \times n}^T \frac{\partial \mathbf{F}_{m \times n}}{\partial x_j} \mathbf{V}_{n \times n}, \quad (14)$$

$$C_{i,j} = \begin{cases} 1 / \left(\sigma_j^2 - \sigma_i^2 \right), & \text{if } i \neq j \\ 0, & \text{otherwise} \end{cases}, \quad (15)$$

where $\circ$ denotes the Hadamard product. $\mathbf{e}_k \in \mathbb{R}^n$ is a unit column vector with the k -th row being one. Also, note that for linear strain tensors with respect to $\mathbf{x}$, the second derivatives of $\mathbf{F}_{m \times n}$ vanish to

zero. Next, using the relation of (10) we obtain

$$\frac{\partial^2 \Psi_{2D}}{\partial \mathbf{f}^2} = \left(\frac{\partial \boldsymbol{\sigma}}{\partial \mathbf{f}} \right)^T \left(\frac{\partial^2 \Psi_{2D}}{\partial \boldsymbol{\sigma}^2} \right) \left(\frac{\partial \boldsymbol{\sigma}}{\partial \mathbf{f}} \right) + \left(\frac{\partial \Psi_{2D}}{\partial \boldsymbol{\sigma}} \right) : \left(\frac{\partial^2 \boldsymbol{\sigma}}{\partial \mathbf{f}^2} \right), \quad (16)$$

where $\mathbf{f}$ is a vectorized deformation gradient such that

$$\mathbf{F}_{3 \times 2} = \begin{bmatrix} f_1 & f_4 \\ f_2 & f_5 \\ f_3 & f_6 \end{bmatrix}. \quad (17)$$

To simplify the problem, we undo the rotation of $\mathbf{F}_{3 \times 2}$ and consider a case

$$\mathbf{F} = \begin{bmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \\ 0 & 0 \end{bmatrix}, \quad \Sigma = \begin{bmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \end{bmatrix}, \quad I_U = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad I_V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad (18)$$

where I_U and I_V are corresponding SVD matrices. With this simplified setting, the first derivative of $\boldsymbol{\sigma}$ is

$$\frac{\partial \boldsymbol{\sigma}}{\partial \mathbf{f}} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}. \quad (19)$$

Using the chain rule and the relations of (12) and (13), we get

$$\frac{\partial^2 \boldsymbol{\sigma}}{\partial f_i \partial f_j} = \left(\frac{\partial I_U}{\partial f_j} \right)^T \frac{\partial \mathbf{F}}{\partial f_i} + I_U^T \frac{\partial \mathbf{F}}{\partial f_i} \left(\frac{\partial I_V}{\partial f_j} \right), \quad (20)$$

$$\frac{\partial I_U}{\partial f_j} = I_U \left(C \circ \left(I_U^T \frac{\partial \mathbf{F}}{\partial f_j} \Sigma + \Sigma \left(\frac{\partial \mathbf{F}}{\partial f_j} \right)^T I_U \right) \right) + \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \frac{\partial \mathbf{F}}{\partial f_j} \Sigma^{-1}, \quad (21)$$

$$\frac{\partial I_V}{\partial f_j} = C \circ \left(\Sigma I_U^T \left(\frac{\partial \mathbf{F}}{\partial f_j} \right) + \left(\frac{\partial \mathbf{F}}{\partial f_j} \right)^T I_U \Sigma \right). \quad (22)$$

By plugging (21) and (22) into (20), we get a pair of Hessians as follows

$$\frac{\partial^2 \sigma_1}{\partial \mathbf{f}^2} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{\sigma_1}{\sigma_1^2 - \sigma_2^2} & 0 & \frac{\sigma_2}{\sigma_1^2 - \sigma_2^2} & 0 & 0 \\ 0 & 0 & \frac{1}{\sigma_1} & 0 & 0 & 0 \\ 0 & \frac{\sigma_2}{\sigma_1^2 - \sigma_2^2} & 0 & \frac{\sigma_1}{\sigma_1^2 - \sigma_2^2} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad (23)$$

$$\frac{\partial^2 \sigma_2}{\partial \mathbf{f}^2} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{\sigma_2}{\sigma_2^2 - \sigma_1^2} & 0 & \frac{\sigma_1}{\sigma_2^2 - \sigma_1^2} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{\sigma_1}{\sigma_2^2 - \sigma_1^2} & 0 & \frac{\sigma_2}{\sigma_2^2 - \sigma_1^2} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1}{\sigma_2} \end{bmatrix}. \quad (24)$$

Substituting (23) and (24) to (16), performing analysis on it, and injecting rotations back, we get a list of eigenmatrices

$$Q_1 = \frac{1}{\sqrt{2}} U_{3 \times 3} \begin{bmatrix} 0 & 1 \\ -1 & 0 \\ 0 & 0 \end{bmatrix} V_{2 \times 2}^T, \quad Q_2 = \frac{1}{\sqrt{2}} U_{3 \times 3} \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{bmatrix} V_{2 \times 2}^T, \quad (25a)$$

$$Q_3 = U_{3 \times 3} \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \end{bmatrix} V_{2 \times 2}^T, \quad Q_4 = U_{3 \times 3} \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix} V_{2 \times 2}^T. \quad (25b)$$

and their corresponding eigenvalues

$$\lambda_1 = \frac{1}{\sigma_1 + \sigma_2} \left(\frac{\partial \Psi_{2D}}{\partial \sigma_1} + \frac{\partial \Psi_{2D}}{\partial \sigma_2} \right), \quad (26a)$$

$$\lambda_2 = \frac{1}{\sigma_1 - \sigma_2} \left(\frac{\partial \Psi_{2D}}{\partial \sigma_1} - \frac{\partial \Psi_{2D}}{\partial \sigma_2} \right), \quad (26b)$$

$$\lambda_3 = \frac{1}{\sigma_1} \left(\frac{\partial \Psi_{2D}}{\partial \sigma_1} \right), \quad (26c)$$

$$\lambda_4 = \frac{1}{\sigma_2} \left(\frac{\partial \Psi_{2D}}{\partial \sigma_2} \right). \quad (26d)$$

The remaining two eigenmatrices (Q_5, Q_6) and their eigenvalues (λ_5, λ_6) can be obtained via another eigenanalysis of a 2×2 matrix. Consider the following Hessian matrix

$$H_{2 \times 2} = \begin{bmatrix} \frac{\partial^2}{\partial \sigma_1^2} & \frac{\partial^2}{\partial \sigma_1 \partial \sigma_2} \\ \text{Sym} & \frac{\partial^2}{\partial \sigma_2^2} \end{bmatrix} \Psi_{2D}, \quad (27)$$

and let $[u_1, u_2]$ and $[v_1, v_2]$ be a pair of eigenvectors of $H_{2 \times 2}$, and λ_u and λ_v be the corresponding eigenvalues. With this setting, the remaining two eigenmatrices and eigenvalues are obtained by

$$Q_5 = U_{3 \times 3} \begin{bmatrix} u_1 & 0 \\ 0 & u_2 \\ 0 & 0 \end{bmatrix} V_{2 \times 2}^T, \quad Q_6 = U_{3 \times 3} \begin{bmatrix} v_1 & 0 \\ 0 & v_2 \\ 0 & 0 \end{bmatrix} V_{2 \times 2}^T, \quad (28)$$

$$\lambda_5 = \lambda_u, \quad \lambda_6 = \lambda_v. \quad (29)$$

Notice that the second eigenvalue results in division by zero if $\sigma_1 = \sigma_2$. Recall that Ψ_{2D} follows the Valanis-Landel hypothesis [2004], which is

$$\frac{\partial \Psi_{2D}}{\partial \sigma_1} \Big|_{\sigma_1=x} = \frac{\partial \Psi_{2D}}{\partial \sigma_2} \Big|_{\sigma_2=x}, \quad (30)$$

where x here denotes an arbitrary scalar. Using the first-order Taylor expansion, we can drop the denominator when $\sigma_1 \approx \sigma_2$

$$\begin{aligned} & \frac{1}{\sigma_1 - \sigma_2} \left(\frac{\partial \Psi_{2D}}{\partial \sigma_1} - \frac{\partial \Psi_{2D}}{\partial \sigma_2} \right) = \\ & \frac{\left(\left(\frac{\partial \Psi_{2D}}{\partial \sigma_1} - \frac{\partial \Psi_{2D}}{\partial \sigma_2} \right)_{\sigma_1 \leftarrow \sigma_2} + (\sigma_1 - \sigma_2) \frac{\partial}{\partial \sigma_1} \left(\frac{\partial \Psi_{2D}}{\partial \sigma_1} - \frac{\partial \Psi_{2D}}{\partial \sigma_2} \right)_{\sigma_1 \leftarrow \sigma_2} \right)}{\sigma_1 - \sigma_2} = \\ & \frac{1}{\sigma_1 - \sigma_2} \left(\frac{\partial \Psi_{2D}}{\partial \sigma_1} - \frac{\partial \Psi_{2D}}{\partial \sigma_2} \right)_{\sigma_1 \leftarrow \sigma_2}^0 + \left(\frac{\partial^2 \Psi_{2D}}{\partial \sigma_1^2} - \frac{\partial^2 \Psi_{2D}}{\partial \sigma_1 \partial \sigma_2} \right)_{\sigma_1 \leftarrow \sigma_2}, \end{aligned} \quad (31)$$

where $\sigma_1 \leftarrow \sigma_2$ denotes a substitution only at evaluation time. Recall that in the above, we symbolically assume $\sigma_1 \approx \sigma_2$, not $\sigma_1 = \sigma_2$

exactly. Hence, $1/(\sigma_1 - \sigma_2)$ does not vanish to infinity. Therefore, when $\sigma_1 \approx \sigma_2$ we can safely use the approximation

$$\frac{1}{\sigma_1 - \sigma_2} \left(\frac{\partial \Psi_{2D}}{\partial \sigma_1} - \frac{\partial \Psi_{2D}}{\partial \sigma_2} \right) \approx \frac{\partial^2 \Psi_{2D}}{\partial \sigma_1^2} - \frac{\partial^2 \Psi_{2D}}{\partial \sigma_1 \partial \sigma_2}. \quad (32)$$

If we assume continuity, this approximation should also hold when $\sigma_1 = \sigma_2$ exactly. We provide a comprehensive Python code (eigsys-2.py) to help readers numerically verify the above analysis.

Other eigenvalues λ_1, λ_3 and λ_4 can technically result in division by zero for extreme cases $\sigma_k = 0$ or $\sigma_j + \sigma_k = 0$, which is possible when an inversion-aware SVD [Sorkine-Hornung and Rabinovich 2017] is employed. These singularities are not unconditionally removable but when Ψ_{2D} is written in terms of two Green-Cauchy invariants $I_1^{2D} = \text{tr}(F^T F) = \sigma_1^2 + \sigma_2^2$ and $I_2^{2D} = \det(F^T F) = \sigma_1^2 \sigma_2^2$ such that

$$\Psi_{2D} = \Psi(I_1^{2D}, I_2^{2D}), \quad (33)$$

they become removable [Stomakhin et al. 2012]. More specifically, $\lambda_1 \dots \lambda_4$ are written as

$$\begin{bmatrix} \lambda_1 \\ \lambda_2 \\ \lambda_3 \\ \lambda_4 \end{bmatrix} = 2 \begin{bmatrix} 1 & \sigma_1 \sigma_2 \\ 1 & -\sigma_1 \sigma_2 \\ 1 & \sigma_1^2 \\ 1 & \sigma_1^2 \end{bmatrix} \begin{bmatrix} \frac{\partial}{\partial I_1^{2D}} \\ \frac{\partial}{\partial I_2^{2D}} \end{bmatrix} \Psi_{2D}. \quad (34)$$

E.1 Strain Limiting Eigenanalysis

When the energy does not have area terms (e.g., $\frac{\partial^2 \Psi_{2D}}{\partial \sigma_1 \partial \sigma_2} = 0$), the eigenanalysis of $H_{2 \times 2}$ becomes trivial, allowing all the eigenmatrices and eigenvalues to be concisely expressed without symbolic eigendecomposition of $H_{2 \times 2}$, given that derivatives of Ψ_{2D} with respect to singular values are known. In this special case, eigen matrices of $\partial^2 \psi_{SL} / \partial F^2$ are all static

$$\begin{aligned} Q_1 &= \frac{1}{\sqrt{2}} U \begin{bmatrix} 0 & 1 \\ -1 & 0 \\ 0 & 0 \end{bmatrix} V^T, \quad Q_2 = \frac{1}{\sqrt{2}} U \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{bmatrix} V^T, \quad Q_3 = U \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \end{bmatrix} V^T, \\ Q_4 &= U \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix} V^T, \quad Q_5 = U \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} V^T, \quad Q_6 = U \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} V^T, \end{aligned} \quad (35)$$

where $\psi_{SL}(\sigma_1, \sigma_2, \sigma_3) = \sum_k \psi_{SL1}(\sigma_k)$ is a strain limiting energy. Its corresponding eigenvalues are

$$\lambda_1 = \frac{1}{\sigma_1 + \sigma_2} \left(\frac{\partial \psi_{SL1}}{\partial \sigma} \Big|_{\sigma=\sigma_1} + \frac{\partial \psi_{SL1}}{\partial \sigma} \Big|_{\sigma=\sigma_2} \right), \quad (36a)$$

$$\lambda_2 = \frac{1}{\sigma_1 - \sigma_2} \left(\frac{\partial \psi_{SL1}}{\partial \sigma} \Big|_{\sigma=\sigma_1} - \frac{\partial \psi_{SL1}}{\partial \sigma} \Big|_{\sigma=\sigma_2} \right), \quad (36b)$$

$$\lambda_3 = \frac{1}{\sigma_1} \frac{\partial \psi_{SL1}}{\partial \sigma} \Big|_{\sigma=\sigma_1}, \quad \lambda_4 = \frac{1}{\sigma_2} \frac{\partial \psi_{SL1}}{\partial \sigma} \Big|_{\sigma=\sigma_2}, \quad (36c)$$

$$\lambda_5 = \frac{\partial^2 \psi_{SL1}}{\partial \sigma^2} \Big|_{\sigma=\sigma_1}, \quad \lambda_6 = \frac{\partial^2 \psi_{SL1}}{\partial \sigma^2} \Big|_{\sigma=\sigma_2}. \quad (36d)$$

One may notice that when $\sigma_1 = \sigma_2$, (36b) results in division by zero, but this case turns out to be the definition of differentiation. Therefore, we use $\lambda_2 = \partial^2 \psi_{SL1} / \partial \sigma^2|_{\sigma=\sigma^*}$, where $\sigma^* = (\sigma_1 + \sigma_2)/2$. We switch to this if $|\sigma_1 - \sigma_2| < \epsilon_{\text{tol}}$ where we use $\epsilon_{\text{tol}} = 10^{-2}$. We note that the above eigenmatrices (not the eigenvalues) have

been previously revealed [Panetta 2020] for invariants in a different context.

F END-TO-END ALGORITHMIC DETAIL

This section aims to provide a clear picture of the end-to-end algorithmic details to help readers implement our method if they choose to do so. Due to this nature, a substantial portion of the material in this section recapitulates previous work. These are clearly not our contribution but are necessary to reiterate for clarity and completeness. A link to the code of our implementation is available in the main document.

Below, we start by presenting the important building blocks of our pipeline by component and then lay them out together in a procedural manner to illustrate our whole pipeline.

F.1 Element-wise Pre-computation

Given a set of vertices and a list of tetrahedra and triangles representing volumetric deformable solids and shells, we pre-compute D^{-1} [Kim and Eberle 2022; Sifakis and Barbic 2012] such that

$$D_{\text{tet}}^{-1} = \begin{bmatrix} \hat{\mathbf{x}}_4 - \hat{\mathbf{x}}_1 & \hat{\mathbf{x}}_3 - \hat{\mathbf{x}}_1 & \hat{\mathbf{x}}_2 - \hat{\mathbf{x}}_1 \end{bmatrix}^{-1}, \quad (37a)$$

$$D_{\text{tri}}^{-1} = \left(\begin{bmatrix} \hat{\mathbf{e}}_x & \hat{\mathbf{e}}_y \end{bmatrix}^T \begin{bmatrix} \hat{\mathbf{x}}_3 - \hat{\mathbf{x}}_1 & \hat{\mathbf{x}}_2 - \hat{\mathbf{x}}_1 \end{bmatrix} \right)^{-1}, \quad (37b)$$

where $\hat{\mathbf{x}}_j$ denotes a 3×1 column vector representing the j -th vertex in the rest pose, constituting a tetrahedron or a triangle. $\hat{\mathbf{e}}_x$ and $\hat{\mathbf{e}}_y$ are two orthogonal unit vectors ($\hat{\mathbf{e}}_x \cdot \hat{\mathbf{e}}_y = 0$) that are perpendicular to the triangle normal, satisfying $\hat{\mathbf{e}}_x \cdot \hat{\mathbf{n}}_{\text{tri}} = 0$ and $\hat{\mathbf{e}}_y \cdot \hat{\mathbf{n}}_{\text{tri}} = 0$, where $\hat{\mathbf{n}}_{\text{tri}}$ is the triangle normal at the rest pose. The dimensions of D_{tet}^{-1} and D_{tri}^{-1} are 3×3 and 2×2 , respectively. A set of these matrices is transferred to the device memory to compute the deformation gradients F_{tet} and F_{tri} later on. Accordingly, we compute the mass of vertices, tetrahedra, and triangles.

For deformable solids, we also maintain a list of triangles representing the solid surfaces, which is used to detect collisions with other deformables at runtime. From this list of triangles, we extract all the unique edges for edge-edge continuous collision detection (CCD) tasks. For shell triangles, we additionally extract hinges [Grinspun et al. 2003], which are used to compute angle-based bending energies.

F.2 Constraint Line Search

Our line search is unconventional: we perform the search using a direction that is stretched by 25% longer than the original. We then check if this point lies beyond the boundary of the admissible region. If it does, we use the corresponding scaling factor α to rescale the *original* search direction and apply it for the actual positional updates.

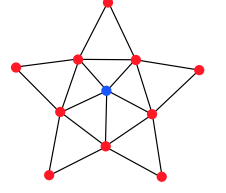
For example, if the original search direction $\mathbf{d}$ points exactly on the admissible boundary, we use $\mathbf{d}/(1.25)$ to update the variables.

If $(1.2)\mathbf{d}$ points exactly on the boundary, we use $(1.2/1.25)\mathbf{d}$. Notice that in the latter case, updating variables with $\mathbf{d}$ does not hit the boundary, but the actual update amounts to $(1.2/1.25)\mathbf{d}$.

F.3 Pre-defined Per-Vertex Index Table

For efficient matrix assembly on the GPU, we collect a set of indices that encode matrix entries we know will be used when embedding elasticity and bending energy potentials.

An example is visualized in the right inset. In this example, the corresponding pre-defined vertices for the blue vertex are shown in red. Once the indices are collected, we sort them by index numbers to enable fast entry insertion at runtime.



F.4 Contact Culling

We employ a Bounding Volume Hierarchy (BVH) and Axis-Aligned Bounding Box (AABB) for collision culling. Specifically, we follow the strategy suggested by Bridson et al. [2002]. The BVH tree is constructed at fixed intervals (every ten video frames in our examples) on the host and transferred to device memory. On the device side, AABBs are arranged according to their depth and reconstructed in parallel from fine to coarse order each time before CCD is dispatched.

With the aid of the contact culling, we collect proximity contacts using edge-edge and point-triangle proximity queries [Andrews et al. 2022]. In the process, we pay careful attention to ensure that duplicated contacts impinging on the same contact point are not counted individually. For instance, both point-triangle and edge-edge proximity queries can yield the same point-point contacts. Such duplication can lead to redundant matrix entry allocations, affecting both performance and the maximum number of tractable contact counts. This is more clearly explained in §F.5.

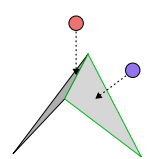
F.5 CCD and Proximity Query

After culling the majority of collision possibilities, we may either perform CCD or find the proximity between nearly touching colliders. For this purpose, we use the Additive Continuous Collision Detection (ACCD) [Li et al. 2021], which we find robust against single-precision floating-point rounding errors. ACCD not only reports the existence of a collision but also provides an accurate (conservative) timing of when the collision occurs. We use this timing (time of impact), as a search direction scaling factor α .

Note that special care is needed prior to ACCD, as described in §F.2. Let $\mathbf{x}(0)$ and $\mathbf{x}(1)$ be the vertices at $t = 0$ and $t = 1$, respectively. If we wish to perform CCD and obtain $0 < \alpha < 1$, we first extrapolate the displacement: $\mathbf{x}^*(1) \leftarrow 1.25(\mathbf{x}(1) - \mathbf{x}(0)) + \mathbf{x}(0)$. We then run CCD to obtain the time of impact between $\mathbf{x}^*(1)$ and $\mathbf{x}(0)$.

A proximity query without duplicates requires special care. For example, in a point-triangle query, let γ_j be a barycentric coordinate representing the closest position $\mathbf{q}_{\text{tri}}$ on the triangle, where $\mathbf{q}_{\text{tri}} = \sum_{j=1}^3 \gamma_j \mathbf{x}_j$. When $0 < \gamma_j < 1$ for all j , this indicates a unique contact point, and thus it is counted as a proximity.

When this is not the case, the proximity may be classified as point-edge or point-point. If we naively continue with edge-edge or point-point attempts without considering adjacent triangles, the proximity may be double-counted. This is illustrated in the right inset, where the blue point is



unique, while the red point is double-counted due to two separate point-triangle queries.

Hence, our first rule is to only consider proximity points where $0 < \gamma_j < 1$ for all j and collect them as unique contacts.

After identifying all unique point-triangle contacts as described above, we proceed to collect unique point-edge proximity contacts. For this purpose, we dispatch point-edge proximity queries.

Let β_j be interpolation coefficients encoding the closest position on the edge such that $\mathbf{q}_{\text{edge}} = \sum_{j=1,2} \beta_j \mathbf{x}_j$. Since we are only interested in point-edge contacts and not point-point ones at this stage, we drop candidates where $\beta_j < 0$ or $\beta_j > 1$ for any j .

Notice that at this stage, the blue point in the inset will still be reported as a possible point-edge proximity, which should not be counted. We exclude such candidates by checking if the point is already reported as a unique contact by neighboring triangles that share the edge of interest. For example, the blue point in the inset may be reported as a proximity point of three edges shown in green. In this case, we check the triangles that contain the green edges, and if the proximity point on the triangle is represented as interpolation from the three vertices.

We extend this approach to exclude point-point proximity duplicates. Similarly, we apply a comparable strategy to collect edge-edge proximity contacts, ensuring no duplicates.

For edge-edge and point-point proximity queries, we need to pay extra attention to situations where two cases refer to the same proximity pair. For a simple case, suppose that we only have two points $\mathbf{p}_1$ and $\mathbf{p}_2$. When we launch the point-point proximity kernel on these two particles, two cases are reported: $(\mathbf{p}_1, \mathbf{p}_2)$ and $(\mathbf{p}_2, \mathbf{p}_1)$. These two pairs obviously must be treated as a single pair. Such duplicates can be simply filtered by comparing their associated index numbers: e.g., $i < j$ for $(\mathbf{p}_i, \mathbf{p}_j)$.

Once all unique contacts are collected, we use the barycentric coordinates β_j and γ_j to construct W_i , which maps vertices to $\mathbf{p}_i - \mathbf{q}_i$. Here, $\mathbf{p}_i$ and $\mathbf{q}_i$ are two nearly touching surface points forming a proximity contact, expressed as a linear combination of mesh vertices.

F.6 Air Dynamics

To mimic a dragging effect by air, we append the following potential to triangle shells

$$\psi_{\text{wind}} = \frac{1}{2} \lambda_{\text{air}} A_{\text{tri}} (\mathbf{n}_{\text{tri}} \cdot (\mathbf{u} - \mathbf{u}_{\text{wind}}))^2, \quad (38)$$

where $\mathbf{u}_{\text{wind}}$ is the ambient wind and λ_{air} is a dragging coefficient. A_{tri} is the triangle area and $\mathbf{n}_{\text{tri}}$ is the triangle normal. Substituting $\mathbf{u} = (\mathbf{x} - \mathbf{x}^{\text{prev}})/\Delta t$, we get

$$\psi_{\text{wind}} = \frac{1}{2\Delta t^2} \lambda_{\text{air}} A_{\text{tri}} (\mathbf{n}_{\text{tri}} \cdot (\mathbf{x} - \mathbf{x}_{\text{wind}}))^2, \quad (39)$$

which turns out to be a weak pin constraint of $\mathbf{x}$ to $\mathbf{x}_{\text{wind}}$ in the direction of $\mathbf{n}_{\text{tri}}$. $\mathbf{x}_{\text{wind}}$ is given by $\mathbf{x}_{\text{wind}} = \mathbf{x}^{\text{prev}} + \Delta t \mathbf{u}_{\text{wind}}$. The value of λ_{air} is chosen depending on the air density, shell mass and thickness, or artistic criterion; though, we recommend $\lambda_{\text{air}} A_{\text{tri}}$ to be at most around the triangle mass to maintain good system conditioning. This is because $m_{\text{tri}}/\Delta t^2$ corresponds to the stiffness of inertia, where m_{tri} is the triangle mass.

For line segments, we simply replaced $(\mathbf{n}_{\text{tri}} \cdot (\mathbf{u} - \mathbf{u}_{\text{wind}}))^2$ with $\|\mathbf{u} - \mathbf{u}_{\text{wind}}\|^2$; thereby treating them as a point-wise dragging force. Since λ_{air} is also set by artistic control, we do not list these values here. They can be found in our GitHub code.

F.7 Strain Limiting Line Search

The violation of strain limiting can be detected by checking if the maximal singular value of the deformation gradient of a triangle, σ_{max} , exceeds an upper bound of $1 + \tau + \hat{\epsilon}$.

Let $\mathbf{x}(0)$ and $\mathbf{x}^*(1)$ be a pair of vertices defined in §F.5, and let $\mathbf{F}_{\text{tri}}^0$ be the deformation gradient at $\mathbf{x}(0)$. Also, let $\sigma_{\text{max}}(\mathbf{F}_{\text{tri}})$ denote the maximal singular value of the deformation gradient $\mathbf{F}_{\text{tri}}$. With this, we formulate strain limiting line search as finding a root of

$$f(\alpha) = \sigma_{\text{max}}(\mathbf{F}_{\text{tri}}^0 + \alpha \Delta \mathbf{F}_{\text{tri}}) - (1 + \tau + \hat{\epsilon}), \quad (40)$$

where $\Delta \mathbf{F}_{\text{tri}} = \mathbf{F}_{\text{tri}}^* - \mathbf{F}_{\text{tri}}^0$. $\mathbf{F}_{\text{tri}}^*$ is the deformation gradient at the state $\mathbf{x}^*(1)$ where the strain limit has been violated. Therefore, $f(0) < 0$ and $f(1) > 0$, indicating that a root exists in the range $0 < \alpha < 1$.

We run a binary search that seeks for $|f(\alpha) + 2\hat{\epsilon}| < \hat{\epsilon}$ (we use $\hat{\epsilon} = 10^{-4}\hat{\epsilon}$) or a maximum number of attempts (e.g., 1024) is reached. We fall back to a constant-scaling reduction: repeat $\alpha_k \leftarrow 0.9\alpha_{k-1}$ until $f(\alpha_k) < 0$ holds, when the binary search fails.

F.8 Contact Derivatives

When appending contact energies to the total energy, we need

$$\sum_i \frac{\partial \psi_{\text{weak}}}{\partial \mathbf{x}} = \sum_i \frac{\partial g_i}{\partial \mathbf{x}} \frac{\partial \psi_{\text{weak}}}{\partial g_i}, \quad (41a)$$

$$\sum_i \frac{\partial^2 \psi_{\text{weak}}}{\partial \mathbf{x}^2} = \sum_i \left(\left(\frac{\partial g_i}{\partial \mathbf{x}} \right)^T \frac{\partial^2 \psi_{\text{weak}}}{\partial g_i^2} \left(\frac{\partial g_i}{\partial \mathbf{x}} \right) + \frac{\partial^2 g_i}{\partial \mathbf{x}^2} \frac{\partial \psi_{\text{weak}}}{\partial g_i} \right). \quad (41b)$$

In the above, we used a relation that ψ_{weak} depends on g_i , and g_i depends on $\mathbf{x}$. Also, $\partial \psi_{\text{weak}} / \partial g_i$ is a scalar. Using a contact normal $\mathbf{n}_i$, we can write $g_i = \mathbf{n}_i \cdot (W_i \mathbf{x})$, where W_i is a contact direction conversion matrix from the main paper. Substituting this to (41a) and (41b) we get

$$\sum_i \frac{\partial \psi_{\text{weak}}}{\partial \mathbf{x}} = \sum_i \left(\frac{\partial \psi_{\text{weak}}}{\partial g_i} \right) \mathbf{n}_i^T W_i, \quad (42a)$$

$$\sum_i \frac{\partial^2 \psi_{\text{weak}}}{\partial \mathbf{x}^2} = \sum_i \left(\frac{\partial^2 \psi_{\text{weak}}}{\partial g_i^2} \right) W_i^T \mathbf{n}_i \mathbf{n}_i^T W_i. \quad (42b)$$

Here, we semi-implicitly fixed $\mathbf{n}$ so that its derivatives do not appear. A similar strategy is proven to be effective [Chen et al. 2024]. Note that the second term in (41b) vanishes to zero since g_i is linear to $\mathbf{x}$. Notice that (42b) is symmetric semi-positive definite by construction; hence, eigenvalue clamping is not needed.

F.9 Hessian and Forces of Isotropic Elastic Energies

Minimizing the elasticity potential using Newton's method requires efficient computation of its first and second derivatives with respect to the mesh vertices $\mathbf{x}$. In doing so, we use the chain rule [Kim and

Eberle 2022]

$$\frac{\partial \Psi}{\partial \mathbf{x}} = \sum_j \frac{\partial f_j}{\partial \mathbf{x}} \frac{\partial \Psi}{\partial f_j}, \quad (43)$$

$$\frac{\partial^2 \Psi}{\partial \mathbf{x}^2} = \sum_{j,k} \frac{\partial^2 \Psi}{\partial f_j \partial f_k} \left(\frac{\partial f_j}{\partial \mathbf{x}} \right) \left(\frac{\partial f_k}{\partial \mathbf{x}} \right)^T, \quad (44)$$

where f_j is a single entry of a deformation gradient $\mathbf{F}$. Note that the above chain rule is valid only when $\partial^2 f_j / \partial \mathbf{x}^2$ vanishes to zero, which is the case for linear tetrahedral and triangular elements. $\mathbf{F}$ is obtained [Kim and Eberle 2022] by

$$\mathbf{F}_{\text{tet}} = \begin{bmatrix} \mathbf{x}_4 - \mathbf{x}_1 & \mathbf{x}_3 - \mathbf{x}_1 & \mathbf{x}_2 - \mathbf{x}_1 \end{bmatrix} D_{\text{tet}}^{-1}, \quad (45a)$$

$$\mathbf{F}_{\text{tri}} = \begin{bmatrix} \mathbf{x}_3 - \mathbf{x}_1 & \mathbf{x}_2 - \mathbf{x}_1 \end{bmatrix} D_{\text{tri}}^{-1}, \quad (45b)$$

where $\mathbf{x}_j$ denotes the j -th vertex in the deformed state. Hence, the computation of $\partial f_j / \partial \mathbf{x}$ becomes trivial. $\partial \Psi / \partial \mathbf{x}$ is obtained by (8).

This reduces the problem of computing $\partial^2 \Psi / \partial \mathbf{x}^2$ to $\partial^2 \Psi / \partial \mathbf{F}^2$. For isotropic energies, where the potential is explicitly expressed by invariants (e.g., Cauchy-Green invariants), this derivative is obtained with the aid of a lookup table listing pre-derived symbolic derivatives of invariants with respect to $\mathbf{F}$. Using knowledge from [Smith et al. 2019], we can symbolically write the eigenmatrices and their eigenvalues, facilitating projection onto the semi-positive symmetric definite space.

In our implementation, we opt for a different strategy. The only difference compared to [Smith et al. 2019] is that we employ singular values instead of invariants. The methods in [Stomakhin et al. 2012; Xu et al. 2015] encounter singularities such as $\sigma_j = \sigma_k$ and requires the eigen decomposition of one 3×3 matrix and three 2×2 matrices.

Our method is mathematically similar to the analysis in [Zhu 2021] and can be considered a variant. However, ours more explicitly presents the eigenmatrices and their eigenvalues. It also eliminates the singularities: singularities where $\sigma_j = \sigma_k$ are unconditionally removed, and those where $\sigma_i + \sigma_j = 0$ are removed when the energy is defined as a function of Cauchy-Green invariants.

In §E, we presented an eigenanalysis of triangular shell elements, but the same analysis extends to tetrahedral solids. Let $\Psi_{3D}(\sigma_1, \sigma_2, \sigma_3)$ be a volumetric elasticity potential with respect to the singular values of $\mathbf{F}_{\text{tet}}$. Hence, Ψ_{3D} is also a function of $\mathbf{F}_{\text{tet}}$. The eigenmatrices of $\partial^2 \Psi_{3D} / \partial \mathbf{F}_{\text{tet}}^2$ are

$$\begin{aligned} Q_1 &= \frac{1}{\sqrt{2}} U_{3 \times 3} \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} V_{3 \times 3}^T, & Q_2 &= \frac{1}{\sqrt{2}} U_{3 \times 3} \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ -1 & 0 & 0 \end{bmatrix} V_{3 \times 3}^T, \\ Q_3 &= \frac{1}{\sqrt{2}} U_{3 \times 3} \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{bmatrix} V_{3 \times 3}^T, & Q_4 &= \frac{1}{\sqrt{2}} U_{3 \times 3} \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} V_{3 \times 3}^T, \\ Q_5 &= \frac{1}{\sqrt{2}} U_{3 \times 3} \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix} V_{3 \times 3}^T, & Q_6 &= \frac{1}{\sqrt{2}} U_{3 \times 3} \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} V_{3 \times 3}^T, \end{aligned} \quad (46)$$

where $U_{3 \times 3}$ and $V_{3 \times 3}$ are rotation matrices obtained by the SVD of $\mathbf{F}_{\text{tet}}$. Their corresponding eigenvalues are

$$\begin{aligned} \lambda_1 &= \frac{1}{\sigma_1 + \sigma_2} \left(\frac{\partial \Psi_{3D}}{\partial \sigma_1} + \frac{\partial \Psi_{3D}}{\partial \sigma_2} \right), & \lambda_2 &= \frac{1}{\sigma_1 + \sigma_3} \left(\frac{\partial \Psi_{3D}}{\partial \sigma_1} + \frac{\partial \Psi_{3D}}{\partial \sigma_3} \right), \\ \lambda_3 &= \frac{1}{\sigma_2 + \sigma_3} \left(\frac{\partial \Psi_{3D}}{\partial \sigma_2} + \frac{\partial \Psi_{3D}}{\partial \sigma_3} \right), & \lambda_4 &= \frac{1}{\sigma_1 - \sigma_2} \left(\frac{\partial \Psi_{3D}}{\partial \sigma_1} - \frac{\partial \Psi_{3D}}{\partial \sigma_2} \right), \\ \lambda_5 &= \frac{1}{\sigma_1 - \sigma_3} \left(\frac{\partial \Psi_{3D}}{\partial \sigma_1} - \frac{\partial \Psi_{3D}}{\partial \sigma_3} \right), & \lambda_6 &= \frac{1}{\sigma_2 - \sigma_3} \left(\frac{\partial \Psi_{3D}}{\partial \sigma_2} - \frac{\partial \Psi_{3D}}{\partial \sigma_3} \right), \end{aligned} \quad (47)$$

where σ_1, σ_2 , and σ_3 denote the singular values of $\mathbf{F}_{\text{tet}}$. The remaining three eigenmatrices and eigenvalues are obtained through an eigen decomposition of a 3×3 Hessian matrix.

$$H_{3 \times 3} = \begin{bmatrix} \frac{\partial^2}{\partial \sigma_1^2} & \frac{\partial^2}{\partial \sigma_1 \partial \sigma_2} & \frac{\partial^2}{\partial \sigma_1 \partial \sigma_3} \\ & \frac{\partial^2}{\partial \sigma_2^2} & \frac{\partial^2}{\partial \sigma_2 \partial \sigma_3} \\ \text{Sym} & & \frac{\partial^2}{\partial \sigma_3^2} \end{bmatrix} \Psi_{3D}. \quad (48)$$

Note that the eigen decomposition is trivial when $H_{3 \times 3}$ is diagonal. Let $\mathbf{u}_{3 \times 1}$, $\mathbf{v}_{3 \times 1}$, and $\mathbf{w}_{3 \times 1}$ be a set of eigenvectors of $H_{3 \times 3}$, and λ_u , λ_v , and λ_w be the corresponding eigenvalues. The remaining three eigenmatrices and eigenvalues are then

$$Q_7 = U_{3 \times 3} \text{diag}(\mathbf{u}_{3 \times 1}) V_{3 \times 3}^T, \quad (49)$$

$$Q_8 = U_{3 \times 3} \text{diag}(\mathbf{v}_{3 \times 1}) V_{3 \times 3}^T, \quad (50)$$

$$Q_9 = U_{3 \times 3} \text{diag}(\mathbf{w}_{3 \times 1}) V_{3 \times 3}^T, \quad (51)$$

$$\lambda_7 = \lambda_u, \quad \lambda_8 = \lambda_v, \quad \lambda_9 = \lambda_w. \quad (52)$$

Readers may notice that $\lambda_{4 \dots 6}$ result in division by zero when $\sigma_j = \sigma_k$. However, similar to (32), these singularities are removable

$$\frac{1}{\sigma_j - \sigma_k} \left(\frac{\partial \Psi_{3D}}{\partial \sigma_j} - \frac{\partial \Psi_{3D}}{\partial \sigma_k} \right) \approx \frac{\partial^2 \Psi_{3D}}{\partial \sigma_j^2} - \frac{\partial^2 \Psi_{3D}}{\partial \sigma_j \partial \sigma_k}. \quad (53)$$

We provide a concise and portable Python code (eigsys-3.py) that numerically verifies the correctness of our analysis, including nearly singular cases where $\sigma_j \approx \sigma_k$. Example elasticity energies tested in the supplementary code (with Lamé and model-specific parameters substituted) are listed below. The code uses a built-in SVD, but our GitHub code employs an inversion-aware SVD [Sorkine-Hornung and Rabinovich 2017].

Model	Expression
ARAP	$\sum_{j=1}^3 (\sigma_j - 1)^2$
Symmetric Dirichlet	$\sum_{j=1}^3 \left(\sigma_j^2 + 1/\sigma_j^2 \right)$
MIPS	$\sum_{j=1}^3 \sigma_j^2 / \sigma_1 \sigma_2 \sigma_3$
Ogden	$\sum_{k=0}^4 \left(\sigma_1^{0.5^k} + \sigma_2^{0.5^k} + \sigma_3^{0.5^k} - 3 \right)$
Yeoh	$\sum_{k=1}^3 \left((\sigma_1^2 + \sigma_2^2 + \sigma_3^2 - 3)^k \right)$

In cases where $\sigma_j + \sigma_k = 0$ such as when elements are squished to a point $\sigma_j = \sigma_k = 0$ or are invertible (only when an inversion-aware SVD [Sorkine-Hornung and Rabinovich 2017] is employed) $\lambda_{1 \dots 3}$

can blow up to infinity. For elasticity energies where Ψ_{3D} is defined in terms of Cauchy-Green invariants,

$$I_1^{3D} = \text{tr}(\mathbf{F}_{\text{tet}}^T \mathbf{F}_{\text{tet}}) = \sigma_1^2 + \sigma_2^2 + \sigma_3^2, \quad (54)$$

$$I_2^{3D} = \frac{1}{2} \left(\left(I_1^{3D} \right)^2 - \|\mathbf{F}_{\text{tet}}^T \mathbf{F}_{\text{tet}}\|^2 \right) = (\sigma_1 \sigma_2)^2 + (\sigma_2 \sigma_3)^2 + (\sigma_3 \sigma_1)^2, \quad (55)$$

$$I_3^{3D} = \det(\mathbf{F}_{\text{tet}}^T \mathbf{F}_{\text{tet}}) = (\sigma_1 \sigma_2 \sigma_3)^2, \quad (56)$$

such that

$$\Psi_{3D} = \Psi(I_1^{3D}, I_2^{3D}, I_3^{3D}), \quad (57)$$

$\lambda_{1\dots 6}$ are written without singularity [Stomakhin et al. 2012]

$$\begin{bmatrix} \lambda_1 \\ \lambda_2 \\ \lambda_3 \\ \lambda_4 \\ \lambda_5 \\ \lambda_6 \end{bmatrix} = 2 \begin{bmatrix} 1 & \sigma_1 \sigma_2 + \sigma_3^2 & \sigma_1 \sigma_2 \sigma_3^2 \\ 1 & \sigma_1 \sigma_3 + \sigma_2^2 & \sigma_1 \sigma_2^2 \sigma_3 \\ 1 & \sigma_2 \sigma_3 + \sigma_1^2 & \sigma_1^2 \sigma_2 \sigma_3 \\ 1 & \sigma_3^2 - \sigma_1 \sigma_2 & -\sigma_1 \sigma_2 \sigma_3^2 \\ 1 & \sigma_2^2 - \sigma_1 \sigma_3 & -\sigma_1 \sigma_2^2 \sigma_3 \\ 1 & \sigma_1^2 - \sigma_2 \sigma_3 & -\sigma_1^2 \sigma_2 \sigma_3 \end{bmatrix} \begin{bmatrix} \frac{\partial}{\partial I_1^{3D}} \\ \frac{\partial}{\partial I_2^{3D}} \\ \frac{\partial}{\partial I_3^{3D}} \end{bmatrix} \Psi_{3D}. \quad (58)$$

Therefore, all the singularities of λ_j are removable in this specific case provided that $\frac{\partial \Psi_{3D}}{\partial I_k}$ does not contain singularities. Unfortunately, the same analogy does not apply to arbitrary energies, such as the co-rotational energy; when $\sigma_j + \sigma_k = 0$, the eigenvalues of $\lambda_{1\dots 3}$ are undefined.

Since we use Newton's method, the eigenvalues are clamped to $\lambda_j \leftarrow \max(0, \lambda_j)$ to ensure semi-positive definiteness. The inclusion of the inertia term ensures that the entire system is eventually positive definite. We use a closed-form eigen decomposition solver for $H_{3 \times 3}$ provided by the Eigen library¹ [Guennebaud et al. 2010]. What needs to be done manually is to symbolically derive $\partial \Psi_{3D} / \partial \sigma_{1\dots 3}$ and $H_{3 \times 3}$. Alternatively, one may resort to automatic differentiation tools [Schmidt et al. 2022], which fully automate all the steps.

F.9.1 Relation to [Smith et al. 2019]. Our eigen system is identical to the one presented by [Smith et al. 2019]. Specifically, the eigenmatrices $Q_{1,2,3}$ correspond to twist modes, and $Q_{4,5,6}$ to flip modes. The equivalence of eigenvalues can be verified using the chain rule $\frac{\partial \Psi}{\partial \sigma_j} = \sum_k \frac{\partial \Psi}{\partial I_k} \frac{\partial I_k}{\partial \sigma_j}$, substituting these into (47), where I_k are the invariants from [Smith et al. 2019]. The equivalence of scaling modes $Q_{7,8,9}$ and their singular values $\lambda_{7,8,9}$ is confirmed by showing that $H_{3 \times 3} = A$, where A is an encoded matrix from [Smith et al. 2019]. We provide a SymPy script (smith2019_equiv.py) that symbolically checks both equivalences.

F.10 Inertia Potential

Following the work of [Martin et al. 2011], we define the inertia trajectory of mesh vertices as the solution to the minimizer

$$\psi_{\text{inertia}}(\Delta t, \mathbf{x}) = \frac{1}{2\Delta t^2} (\mathbf{x} - \mathbf{z})^T M (\mathbf{x} - \mathbf{z}), \quad (59)$$

$$\mathbf{z} = \mathbf{x}_{\text{prev}} + \Delta t \mathbf{u}_{\text{prev}} + \Delta t^2 \mathbf{g}_{\text{gravity}}, \quad (60)$$

where M is a diagonal mass matrix, Δt is the step size, $\mathbf{x}_{\text{prev}}$ represents the vertices at the previous step, $\mathbf{u}_{\text{prev}}$ denotes the velocity at

the previous step, and $\mathbf{g}_{\text{gravity}}$ is the gravity vector. The first and second derivatives of (59) with respect to $\mathbf{x}$ are straightforward.

F.11 Elasticity Potential

Given isotropic constitutive energy density functions Ψ_{2D} and Ψ_{3D} , we define the elasticity energy as

$$\psi_{\text{elastic}}(\mathbf{x}) = \sum_k A_k \Psi_{2D}(\sigma(\mathbf{F}_{\text{tri}}^k(\mathbf{x}))) + \sum_k V_k \Psi_{3D}(\sigma(\mathbf{F}_{\text{tet}}^k(\mathbf{x}))), \quad (61)$$

where A_k and V_k are the area of the triangle and the volume of the tetrahedron, respectively. $\sigma(\mathbf{F}_{\text{tri}}^k(\mathbf{x}))$ encodes the singular values computed from the deformation gradient of the k -th element with respect to vertices $\mathbf{x}$. The (semi-)eigenanalysis of Ψ_{2D} and Ψ_{3D} are provided in §E and §F.9 to facilitate obtaining the first and second derivatives of (61) with respect to $\mathbf{x}$. We use a dihedral angle-based bending energy [Grinspun et al. 2003] and employ a Gauss-Newton approximation for its second derivatives, as suggested by [Kim and Eberle 2022].

For shells, we also employ the method of [Baraff and Witkin 1998], though it is not isotropic and therefore its eigensystem requires dedicated analysis [Kim 2020]. For solid simulation, we use the Stable Neo-Hookean model [Smith et al. 2018]. For clarity, we provide $\partial \Psi_{\text{tet}} / \partial \sigma_j$ and the corresponding $H_{3 \times 3}$ in §F.14.

For segments, which are not covered in Algorithm 3, we employ Hooke's law $\psi(l) = \frac{\kappa}{2} (l - l_0)^2$, where l is the length of an edge and l_0 is the rest length.

F.12 Boundary Conditions

Boundary condition potentials are described in the main document; here, we detail implementation-specific choices. For static pin constraints, we perform a line search to keep a vertex pinned to the target within a tolerance.

On the other hand, we do not perform line search for moving pins, as it more strictly limits the feasible step size. Even without this line search, side effects were not noticeable. This is only the case with our elasticity-inclusive barrier stiffness; without it, pins might noticeably deviate from the desired anchor points.

For mesh-represented solid walls, we eliminate the degrees of freedom when the mesh is static and perform a line search to ensure that other colliders are pushed out. When the mesh deforms, we treat it as collidable surfaces, like shells, and apply moving pin constraints to its vertices.

F.13 Our End-to-End Simulation Pipeline

With the materials presented in the main document and all the components introduced above, we can build a complete pipeline for our elasticity solver with contact resolution. A pseudocode illustration is provided in Algorithm 3.

F.13.1 Initialization. We start by pre-computing mesh-related data before entering the simulation loop. This includes

- D_{tri}^{-1} and D_{tet}^{-1} for triangle and tetrahedral elements (§F.1).
- List of neighborhood indices per vertex for efficient matrix assembly (§F.3).
- BVH tree and AABBs enclosing mesh elements (§F.4).

¹https://eigen.tuxfamily.org/dox/classEigen_1_1SelfAdjointEigenSolver.html

Algorithm 3 Our End-to-End Algorithm

```

1: function PREPARATION                                ▷ §F.13.1
2:   Compute  $D_{\text{tri}}^{-1}$  and  $D_{\text{tet}}^{-1}$  for triangles and tets    ▷ §F.1
3:   Compute index table for efficient matrix assembly    ▷ §F.3
4:   Build BVH tree and compute AABB                    ▷ §F.4
5: end function
6: function RUN_SIMULATION( $\Delta t, \mathbf{x}, \mathbf{u}$ )    ▷  $\mathbf{x}$ : vertices  $\mathbf{u}$ : velocity
7:    $t, \mathbf{x}_{\text{prev}} \leftarrow 0, \mathbf{x}$                 ▷  $\mathbf{x}_{\text{prev}}$ : previous position by  $\mathbf{u}$ 
8:   while  $t < t_{\text{max}}$  do                        ▷ Enter into simulation loop
9:      $\mathbf{x}, \Delta t_{\text{prev}} \leftarrow \text{advance\_step}(\Delta t, \mathbf{x}, \mathbf{x}_{\text{prev}}, \mathbf{u})$     ▷ §15
10:     $\mathbf{u} \leftarrow (\mathbf{x} - \mathbf{x}_{\text{prev}}) / \Delta t_{\text{prev}}$     ▷ Compute velocity
11:     $t, \mathbf{x}_{\text{prev}} \leftarrow t + \Delta t_{\text{prev}}, \mathbf{x}$     ▷ Increment time
12:    Rebuild BVH tree if needed    ▷ Every 10 video frames
13:   end while
14: end function
15: function ADVANCE_STEP( $\Delta t, \mathbf{x}, \mathbf{x}_{\text{prev}}, \mathbf{u}$ )    ▷ §F.13.2
16:    $\beta \leftarrow 0$ 
17:   while  $\beta < \beta_{\text{max}}$  do                        ▷  $\beta_{\text{max}} = 0.25$ 
18:      $\mathbf{x}, \alpha \leftarrow \text{inner\_step}(\Delta t, \mathbf{x}, \mathbf{x}_{\text{prev}})$     ▷ Newton's step
19:      $\beta \leftarrow \beta + (1 - \beta)\alpha$                 ▷ Advance substep
20:   end while
21:    $\mathbf{x}, \_ \leftarrow \text{inner\_step}(\beta\Delta t, \mathbf{x}, \mathbf{x}_{\text{prev}})$     ▷ Error reduction pass
22:   return  $\mathbf{x}, \beta\Delta t$     ▷ New position and actual advanced step
23: end function
24: function INNER_STEP( $\Delta t, \mathbf{x}, \mathbf{x}_{\text{prev}}$ )    ▷ §F.13.3
25:    $H_1, \mathbf{f}_1 \leftarrow \text{Inertia hess and force}$     ▷ §F.10
26:   for all triangle  $j$  and tet element  $k$  do    ▷ Also segments
27:      $H_2^j, \mathbf{f}_2^j \leftarrow \text{Air hess and force using } \mathbf{x}_{\text{prev}}$     ▷ §F.6
28:     Compute  $\mathbf{F}_{\text{tri}}^j$  and  $\mathbf{F}_{\text{tet}}^k$  using  $D_{\text{tri}}^{-1}$  and  $D_{\text{tet}}^{-1}$     ▷ §F.9
29:      $H_3^j, \mathbf{f}_3^j \leftarrow \text{Shell elasticity hess and force using } \mathbf{F}_{\text{tri}}^j$     ▷ §E
30:      $H_4^k, \mathbf{f}_4^k \leftarrow \text{Tet elasticity hess and force using } \mathbf{F}_{\text{tet}}^k$     ▷ §F.9
31:   end for
32:    $H_5, \mathbf{f}_5 \leftarrow \sum_j (H_2^j + H_3^j) + \sum_k (H_4^k + \mathbf{f}_2^j + \mathbf{f}_3^j) + \sum_k \mathbf{f}_4^k$ 
33:   for all proximity contact  $k$  do ▷ BVH tree traversal §F.13.5
34:      $H_6^k, \mathbf{f}_6^k \leftarrow \text{Contact hess and force using } H_5$     ▷ §F.8, §D
35:      $H_7^k, \mathbf{f}_7^k \leftarrow \text{Friction hess/force using } \mathbf{f}_6^k \text{ and } \mathbf{x}_{\text{prev}}$ 
36:   end for
37:    $H_8, \mathbf{f}_8 \leftarrow \sum_k (H_6^k + H_7^k), \sum_k (\mathbf{f}_6^k + \mathbf{f}_7^k)$ 
38:    $H_9, \mathbf{f}_9 \leftarrow \text{Strain limiting hess and force using } H_5$     ▷ §E
39:    $H_{10}, \mathbf{f}_{10} \leftarrow \text{Boundary condition hess/force using } H_5$     ▷ §F.12
40:    $H \leftarrow H_1 + H_2 + H_5 + H_8 + H_9 + H_{10}$     ▷ §F.13.4
41:    $\mathbf{f} \leftarrow \mathbf{f}_1 + \mathbf{f}_2 + \mathbf{f}_5 + \mathbf{f}_8 + \mathbf{f}_9 + \mathbf{f}_{10}$     ▷ §F.13.4
42:    $\mathbf{d} \leftarrow H^{-1}\mathbf{f}$     ▷ Using  $3 \times 3$  block Jacobi PCG
43:    $\mathbf{x}^* \leftarrow \mathbf{x} + (1.25)\mathbf{d}$     ▷ Extended search direction §F.2
44:   Update_AABB( $\mathbf{x}, \mathbf{x}^*$ )    ▷ §F.4
45:    $\alpha_{\text{toi}} \leftarrow \text{CCD\_Line\_Search}(\mathbf{x}, \mathbf{x}^*)$     ▷ ACCD (§F.5, §F.13.5)
46:    $\alpha_{\text{SL}} \leftarrow \text{Strain\_Limit\_Line\_Search}(\mathbf{x}, \mathbf{x}^*)$     ▷ §F.7
47:    $\alpha_{\text{boundary}} \leftarrow \text{Boundary\_Cond\_Line\_Search}(\mathbf{x}, \mathbf{x}^*)$     ▷ §F.12
48:    $\alpha \leftarrow \min(\alpha_{\text{toi}}, \alpha_{\text{SL}}, \alpha_{\text{boundary}})$     ▷ Most conservative  $\alpha$ 
49:   return  $\mathbf{x} + \alpha\mathbf{d}, \alpha$     ▷ New position and substep used
50: end function

```

We also pre-compute mesh-dependent data, such as mass, area, and volumes. All this data, along with the associated mesh structure,

is transferred to device memory. Additionally, we allocate device memory to accommodate diagonal 3×3 and contact matrices.

F.13.2 Advancing a Step. We advance a step by Δt by describing the motion of $\mathbf{x}$ as the solution to the minimization problem

$$\psi(\mathbf{x}) = \psi_{\text{inertia}}(\Delta t, \mathbf{x}) + \psi_{\text{elastic}}(\mathbf{x}) + \psi_{\text{weak}}(\mathbf{x}), \quad (62)$$

where $\psi_{\text{weak}}(\mathbf{x})$ is a barrier potential introduced in the main document. We define $\psi_{\text{dyn}}(\Delta t, \mathbf{x})$ in the main document as $\psi_{\text{dyn}}(\Delta t, \mathbf{x}) = \psi_{\text{inertia}}(\Delta t, \mathbf{x}) + \psi_{\text{elastic}}(\mathbf{x})$.

We minimize (62) with respect to $\mathbf{x}$ using a customized Newton's method as detailed in §C (Lines from 17 to 21). The actual advanced step amounts to $\beta\Delta t$.

F.13.3 Our Newton's Step. Our Newton's step (Line 24) begins by assembling the first and second derivatives of the total energy, which includes a range of potentials such as inertia, elasticity, contact, stitch, air dynamics, strain limiting, and boundary conditions (Lines from 25 to 39). Next, using the collected forces $\mathbf{f}_1 \dots \mathbf{f}_{10}$ and the Hessian matrices $H_1 \dots H_{10}$, we compute the search direction. When the inertia potential is the only driving force, the search direction points exactly to $\mathbf{z}$, where $\mathbf{z}$ is the inertia goal position.

Therefore, when solving the linear system for $\mathbf{d}$ at Line 42, we warm start with $\mathbf{z} - \mathbf{x}$. Although we have not tried it yet, adopting a collision-aware warm start [Lan et al. 2023] might lead to noticeable performance improvements. Finally, a line search is performed (Lines from 45 to 48) to ensure that the updated position remains within the admissible region, meaning that no constraint, including penetration, is violated.

F.13.4 Matrix Assembly. In Algorithm 3, we illustrated that individual Hessians and forces are separately collected and merged at Lines 40 and 41. In our actual implementation, we use a global matrix and a force vector that are defined before the Hessian and force collection begins. These are directly assembled into the global matrix and the vector array. When assembling a matrix entry, we first attempt a diagonal 3×3 and an index-table-fixed matrices (§F.3) and if missed, we continue to the contact matrix for efficient matrix fill-ins.

F.13.5 BVH Tree Traversal and the Size of AABB. BVH tree traversal occurs twice during a single Newton's step: once for contact proximity collection at Line 33 and again for CCD at Line 45. To ensure accurate pruning of unlikely proximities, the AABBs must fully enclose the referenced elements in a hierarchical manner.

This means that the AABBs must be updated whenever elements move or distort in the domain space. In contrast, the BVH tree does not need frequent re-computation; although its efficiency may deteriorate, it remains valid [Bridson et al. 2002].

Therefore, after obtaining the search direction at Line 42, we update the AABBs using both $\mathbf{x}$ and $\mathbf{x}^*$ from Line 43. For example, let $\mathbf{x}_1, \mathbf{x}_2$, and $\mathbf{x}_3$ be three vertices from $\mathbf{x}$ of a triangle, and $\mathbf{x}_1^*, \mathbf{x}_2^*$, and $\mathbf{x}_3^*$ be three vertices from $\mathbf{x}^*$ of the same triangle. We adjust the AABB for the triangle to enclose all of $\mathbf{x}_1 \dots \mathbf{x}_3$ and $\mathbf{x}_1^* \dots \mathbf{x}_3^*$, adding a margin of $\hat{g}$ (contact gap).

This AABB remains valid for proximity contact queries at Line 33 because the vertices at this stage are interpolation $(1 - \alpha)\mathbf{x} + \alpha\mathbf{x}^*$. Thus, another AABB rebuild is not needed before Line 33. As noted in §F.4, the BVH tree is rebuilt every 10 video frames (Line 12).

F.14 $\frac{\partial \Psi_{\text{tet}}}{\partial \sigma}$ and $H_{3 \times 3}$ of Stable Neo-Hookean Model

Here we provide $\partial \Psi_{\text{tet}} / \partial \sigma$ and $H_{3 \times 3}$ for the Stable Neo-Hookean elasticity model [Smith et al. 2018]. The energy density potential is

$$\Psi_{\text{SNH}} = \frac{\lambda}{2} (\sigma_1 \sigma_2 \sigma_3 - 1)^2 - \mu (\sigma_1 \sigma_2 \sigma_3 - 1) + \frac{\mu}{2} (\sigma_1^2 + \sigma_2^2 + \sigma_3^2 - 3). \quad (63)$$

$\partial \Psi_{\text{SNH}} / \partial \sigma$ is

$$\frac{\partial \Psi_{\text{SNH}}}{\partial \sigma} = \begin{bmatrix} \lambda \sigma_2 \sigma_3 (\sigma_1 \sigma_2 \sigma_3 - 1) - \mu \sigma_2 \sigma_3 + \mu \sigma_1 \\ \lambda \sigma_1 \sigma_3 (\sigma_1 \sigma_2 \sigma_3 - 1) - \mu \sigma_1 \sigma_3 + \mu \sigma_2 \\ \lambda \sigma_1 \sigma_2 (\sigma_1 \sigma_2 \sigma_3 - 1) - \mu \sigma_1 \sigma_2 + \mu \sigma_3 \end{bmatrix}. \quad (64)$$

$H_{3 \times 3}$ is

$$\begin{bmatrix} \lambda \sigma_2^2 \sigma_3^2 + \mu & \sigma_3 (2\lambda \sigma_1 \sigma_2 \sigma_3 - \lambda - \mu) & \sigma_2 (2\lambda \sigma_1 \sigma_2 \sigma_3 - \lambda - \mu) \\ \sigma_3 (2\lambda \sigma_1 \sigma_2 \sigma_3 - \lambda - \mu) & \lambda \sigma_1^2 \sigma_3^2 + \mu & \sigma_1 (2\lambda \sigma_1 \sigma_2 \sigma_3 - \lambda - \mu) \\ \sigma_2 (2\lambda \sigma_1 \sigma_2 \sigma_3 - \lambda - \mu) & \sigma_1 (2\lambda \sigma_1 \sigma_2 \sigma_3 - \lambda - \mu) & \lambda \sigma_1^2 \sigma_2^2 + \mu \end{bmatrix} \quad (65)$$

REFERENCES

- Sheldon Andrews, Kenny Erleben, and Zachary Ferguson. 2022. Contact and Friction Simulation for Computer Graphics. In *ACM SIGGRAPH 2022 Courses* (Vancouver, British Columbia, Canada) (SIGGRAPH '22). Association for Computing Machinery, New York, NY, USA, Article 3, 172 pages. <https://doi.org/10.1145/3532720.3535640>
- David Baraff and Andrew Witkin. 1998. Large Steps in Cloth Simulation. In *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '98)*. Association for Computing Machinery, New York, NY, USA, 43–54. <https://doi.org/10.1145/280814.280821>
- Robert Bridson, Ronald Fedkiw, and John Anderson. 2002. Robust Treatment of Collisions, Contact and Friction for Cloth Animation. *ACM Trans. Graph.* 21, 3 (July 2002), 594–603. <https://doi.org/10.1145/566654.566623>
- Anka He Chen, Ziheng Liu, Yang Yin, and Cem Yuksel. 2024. Vertex Block Descent. *ACM Transactions on Graphics (TOG)* 116 (2024).
- Gabriel Cirio, Jorge Lopez-Moreno, David Miraut, and Miguel A. Otaduy. 2014. Yarn-Level Simulation of Woven Cloth. *ACM Trans. Graph.* 33, 6, Article 207 (November 2014), 11 pages. <https://doi.org/10.1145/2661229.2661279>
- Gabriel Cirio, Jorge Lopez-Moreno, and Miguel A. Otaduy. 2017. Yarn-Level Cloth Simulation with Sliding Persistent Contacts. *IEEE Trans. Vis. Comput. Graph.* 23, 2 (2017), 1152–1162. <https://doi.org/10.1109/TVCG.2016.2592908>
- Eitan Grinspun, Anil N. Hirani, Mathieu Desbrun, and Peter Schröder. 2003. Discrete Shells. In *Proceedings of the 2003 ACM SIGGRAPH/Eurographics Symposium on Computer Animation* (San Diego, California) (SCA '03). Eurographics Association, Goslar, DEU, 62–67.
- Gaël Guennebaud, Benoît Jacob, et al. 2010. Eigen v3. <http://eigen.tuxfamily.org>.
- David Harmon, Etienne Vouga, Rasmus Tamstorf, and Eitan Grinspun. 2008. Robust Treatment of Simultaneous Collisions. *ACM Trans. Graph.* 27, 3 (aug 2008), 1–4. <https://doi.org/10.1145/1360612.1360622>
- Kemeng Huang, Floyd Chitalu, Huancheng Lin, and Taku Komura. 2024. GIPC: Fast and stable Gauss-Newton optimization of IPC barrier energy. arXiv:2308.09400 [cs.GR]
- Jonathan M. Kaldor, Doug L. James, and Steve Marschner. 2010. Efficient Yarn-Based Cloth with Adaptive Contact Linearization. *ACM Trans. Graph.* 29, 4, Article 105 (July 2010), 10 pages. <https://doi.org/10.1145/1778765.1778842>
- Theodore Kim. 2020. *A Finite Element Formulation of Baraff-Witkin Cloth*. Eurographics Association, Goslar, DEU. <https://doi.org/10.1111/cgf.14111>
- Theodore Kim and David Eberle. 2022. Dynamic Deformables: Implementation and Production Practicalities (Now with Code!). In *ACM SIGGRAPH 2022 Courses* (Vancouver, British Columbia, Canada) (SIGGRAPH '22). Association for Computing Machinery, New York, NY, USA, Article 7, 259 pages. <https://doi.org/10.1145/3532720.3535628>
- Lei Lan, Minchen Li, Chenfanfu Jiang, Huamin Wang, and Yin Yang. 2023. Second-Order Stencil Descent for Interior-Point Hyperelasticity. *ACM Trans. Graph.* 42, 4, Article 108 (jul 2023), 16 pages. <https://doi.org/10.1145/3592104>
- Minchen Li, Zachary Ferguson, Teseo Schneider, Timothy Langlois, Denis Zorin, Daniele Panozzo, Chenfanfu Jiang, and Danny M. Kaufman. 2020. Incremental Potential Contact: Intersection- and Inversion-free Large Deformation Dynamics. *ACM Trans. Graph. (SIGGRAPH)* 39, 4, Article 49 (2020).
- Minchen Li, Danny M. Kaufman, and Chenfanfu Jiang. 2021. Codimensional Incremental Potential Contact. *ACM Trans. Graph. (SIGGRAPH)* 40, 4, Article 170 (2021).
- Sebastian Martin, Bernhard Thomaszewski, Eitan Grinspun, and Markus Gross. 2011. Example-Based Elastic Materials. *ACM Trans. Graph.* 30, 4, Article 72 (July 2011), 8 pages. <https://doi.org/10.1145/2010324.1964967>
- Rahul Narain, Armin Samii, and James F. O'Brien. 2012. Adaptive Anisotropic Remeshing for Cloth Simulation. *ACM Trans. Graph.* 31, 6, Article 152 (November 2012), 10 pages. <https://doi.org/10.1145/2366145.2366171>
- Julian Panetta. 2020. Analytic Eigensystems for Isotropic Membrane Energies. arXiv:2008.10698 [math.NA]
- Patrick Schmidt, Janis Born, David Bommes, Marcel Campen, and Leif Kobbelt. 2022. TinyAD: Automatic Differentiation in Geometry Processing Made Simple. *Computer Graphics Forum* 41, 5 (2022).
- Andrew Selle, Jonathan Su, Geoffrey Irving, and Ronald Fedkiw. 2009. Robust High-Resolution Cloth Using Parallelism, History-Based Collisions, and Accurate Friction. *IEEE Trans. Vis. Comput. Graph.* 15, 2 (2009), 339–350. <https://doi.org/10.1109/TVCG.2008.79>
- Eftychios Sifakis and Jernej Barbic. 2012. FEM Simulation of 3D Deformable Solids: A Practitioner's Guide to Theory, Discretization and Model Reduction. In *ACM SIGGRAPH 2012 Courses* (Los Angeles, California) (SIGGRAPH '12). Association for Computing Machinery, New York, NY, USA, Article 20, 50 pages. <https://doi.org/10.1145/2343483.2343501>
- Breannan Smith, Fernando De Goes, and Theodore Kim. 2018. Stable Neo-Hookean Flesh Simulation. *ACM Trans. Graph.* 37, 2, Article 12 (March 2018), 15 pages. <https://doi.org/10.1145/3180491>
- Breannan Smith, Fernando De Goes, and Theodore Kim. 2019. Analytic Eigensystems for Isotropic Distortion Energies. *ACM Trans. Graph.* 38, 1, Article 3 (February 2019), 15 pages. <https://doi.org/10.1145/3241041>
- Olga Sorkine-Hornung and Michael Rabinovich. 2017. Least-squares rigid motion using svd. *Computing* 1, 1 (2017), 1–5.

Table 1. Timing breakdown (formatted as Average of all / Average of 10 Highest) and important scene numbers of our examples. #N refers to the average of Newton's steps. $\bar{\mu}, \bar{\rho}, \bar{\lambda}$ refer to Young's modulus, density and the Poisson's ratio.

Scene	Mat Assembly	PCG	Line Search	Time Per Step	Per Frame	Contact Count	Step Size
§4.3	12ms (18ms)	19ms (47ms)	1ms (7ms)	143ms (1.18s)	891ms (6.85s)	5.94K (8.49K)	10.0ms
§4.4	17ms (45ms)	2ms (9ms)	2ms (7ms)	94ms (323ms)	2.29s (5.86s)	1.60K (6.29K)	2.0ms
§4.6T	167ms (196ms)	234ms (344ms)	12ms (15ms)	13.30s (15.91s)	22.21s (30.86s)	1.02M (1.20M)	10.0ms
§4.6B	17ms (21ms)	8ms (135ms)	1ms (2ms)	3.24s (7.92s)	10.78s (23.77s)	7.34K (8.08K)	10.0ms
§4.8	135ms (217ms)	22ms (138ms)	18ms (30ms)	507ms (3.02s)	10.37s (79.08s)	243.86K (497.97K)	1.0ms
§4.10	14ms (20ms)	37ms (78ms)	1ms (2ms)	621ms (2.77s)	34.71s (118s)	1.01K (6.43K)	1.0ms
§4.11	21ms (43ms)	7ms (13ms)	2ms (6ms)	403ms (728ms)	882ms (2.44s)	7.30K (13.75K)	10.0ms
§5.2	5.14s (7.24s)	13.70s (21.66s)	751ms (1.16s)	39.35s (58.16s)	66.34s (116s)	32.71M (53.85M)	10.0ms
§5.3	443ms (1.39s)	60ms (386ms)	34ms (134ms)	4.88s (15.88s)	111s (320s)	2.68M (8.69M)	1.0ms
§5.4	79ms (107ms)	14ms (21ms)	7ms (10ms)	675ms (1.28s)	15.94s (23.86s)	225.10K (343.65K)	2.5ms
§5.5L	782ms (7.77s)	1.14s (2.16s)	124ms (1.52s)	8.35s (47.29s)	246s (1124s)	136.44K (187.39K)	1.0ms
§5.8	5ms (5ms)	47ms (66ms)	0ms (0ms)	1.89s (2.44s)	3.15s (4.74s)	134 (210)	10.0ms
§5.9	8ms (11ms)	10ms (24ms)	0ms (1ms)	99ms (199ms)	873ms (1.22s)	33.65K (39.87K)	10.0ms
§5.11	12ms (32ms)	10ms (32ms)	1ms (4ms)	121ms (639ms)	5.90s (17.21s)	5.52K (47.45K)	1.0ms
§5.13	83ms (221ms)	142ms (295ms)	21ms (47ms)	1.15s (8.75s)	7.83s (68.63s)	1.87K (11.97K)	2.5ms

Scene	#Vert	Shell ($\bar{\mu}, \bar{\rho}$ (kg/m ³), $\bar{\lambda}$)	Tet ($\bar{\mu}, \bar{\rho}$ (kg/m ³), $\bar{\lambda}$)	Mass Rat	Dimension (m)	$\hat{g}$	$\tau + \hat{\epsilon}$	#N
§4.3	28.47K	5 KPa, 10^2 , 0.25	N/A	N/A	$1.28 \times 1.50 \times 0.34$	0.50mm	10%	2.1
§4.4	57.50K	1 MPa, 10^3 , 0.25	N/A	N/A	$23.87 \times 7.96 \times 7.31$	1.00mm	10%	2.1
§4.6T	315.37K	100 KPa, 10^3 , 0.25	N/A	N/A	$1.40 \times 1.40 \times 1.10$	0.50mm	5%	32.0
§4.6B	98.29K	N/A	6 MPa, 10^3 , 0.45	N/A	$1.46 \times 0.19 \times 1.52$	1.00mm	N/A	32.0
§4.8	295.14K	100 KPa, 10^3 , 0.25	50 MPa, 10×10^4 , 0.35	43.06	$2.79 \times 0.65 \times 2.79$	1.00mm	5%	1.7
§4.10	95.94K	10 MPa, 10^3 , 0.25	3 MPa, 6×10^3 , 0.35	32.91	$2.50 \times 2.00 \times 2.50$	1.00mm	1%	7.0
§4.11	48.65K	50 KPa, 10^3 , 0.25	N/A	N/A	$4.67 \times 2.00 \times 2.00$	1.00mm	5%	8.0
§5.2	3.21M	10 MPa, 10^3 , 0.25	N/A	N/A	$3.50 \times 1.80 \times 1.80$	1.75mm	N/A	1.0
§5.3	600.77K	100 KPa, 10^3 , 0.25	25 MPa, 5×10^4 , 0.35	41.24	$2.83 \times 1.60 \times 2.83$	1.00mm	5%	8.0
§5.4	202.00K	100 KPa, 10^3 , 0.25	N/A	N/A	$2.00 \times 0.08 \times 2.00$	1.00mm	5%	4.4
§5.5L	4.00M	N/A	150 KPa, 10^2 , 0.45	N/A	$1.00 \times 5.40 \times 1.00$	0.25mm	N/A	1.6
§5.8	37.86K	N/A	500 KPa, 10^3 , 0.35	N/A	$0.73 \times 0.67 \times 0.88$	1.00mm	N/A	32.0
§5.9	28.56K	N/A	N/A	N/A	$1.01 \times 1.04 \times 0.02$	1.00mm	N/A	1.8
§5.11	58.08K	100 KPa, 10^3 , 0.25	N/A	N/A	$2.00 \times 2.00 \times 0.00$	1.00mm	1%	2.5
§5.13	380.67K	N/A	10 KPa, 10.0, 0.4	N/A	$3.48 \times 7.00 \times 4.00$	1.00mm	N/A	1.8

Alexey Stomakhin, Russell Howes, Craig Schroeder, and Joseph M. Teran. 2012. Energetically Consistent Invertible Elasticity. In *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation* (Lausanne, Switzerland) (SCA '12). Eurographics Association, Goslar, DEU, 25–32.

Min Tang, Huamin Wang, Le Tang, Ruofeng Tong, and Dinesh Manocha. 2016. CAMA: Contact-Aware Matrix Assembly with Unified Collision Handling for GPU-based Cloth Simulation. *Computer Graphics Forum* 35, 2 (2016), 511–521. <https://doi.org/10.1111/cgf.12851> arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.12851

Min Tang, tongtong wang, Zhongyuan Liu, Ruofeng Tong, and Dinesh Manocha. 2018. I-Cloth: Incremental Collision Handling for GPU-Based Interactive Cloth Simulation. 37, 6, Article 204 (December 2018), 10 pages. <https://doi.org/10.1145/3272127.3275005>

James Townsend. 2016. Differentiating the Singular Value Decomposition. (4 2016). <https://j-town.github.io/papers/svd-derivative.pdf>

K. C. Valanis and R. F. Landel. 2004. The Strain-Energy Function of a Hyperelastic Material in Terms of the Extension Ratios. *Journal of Applied Physics* 38, 7 (06 2004), 2997–3002. <https://doi.org/10.1063/1.1710039> arXiv:https://pubs.aip.org/aip/jap/article-pdf/38/7/2997/7404874/2997_1_online.pdf

Tianyu Wang, Jiong Chen, Dongping Li, Xiaowei Liu, Huamin Wang, and Kun Zhou. 2023. Fast GPU-Based Two-Way Continuous Collision Handling. *ACM Trans. Graph.* 42, 5, Article 167 (jul 2023), 15 pages. <https://doi.org/10.1145/3604551>

Thomas Wolf, Victor Cornillère, and Olga Sorkine-Hornung. 2021. Physically-based Book Simulation with Freeform Developable Surfaces. *Computer*

Graphics Forum 40, 2 (2021), 449–460. <https://doi.org/10.1111/cgf.142646> arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.142646

Botao Wu, Zhendong Wang, and Huamin Wang. 2022. A GPU-Based Multilevel Additive Schwarz Preconditioner for Cloth and Deformable Body Simulation. *ACM Trans. Graph.* 41, 4, Article 63 (jul 2022), 14 pages. <https://doi.org/10.1145/3528223.3530085>

Hongyi Xu, Funshing Sin, Yufeng Zhu, and Jernej Barbic. 2015. Nonlinear Material Design Using Principal Stretches. *ACM Trans. Graph.* 34, 4, Article 75 (July 2015), 11 pages. <https://doi.org/10.1145/2766917>

Yufeng Zhu. 2021. Eigen Space of Mesh Distortion Energy Hessian. arXiv:2103.08141 [cs.GR] <https://arxiv.org/abs/2103.08141>

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009

```

1 from math import log as ln
2
3 # n'th derivative of a simple quadratic energy
4 def momentum(y,n):
5     return [y**2,2*y,2][n]/2
6
7 # n'th derivative of logarithmic barrier energy
8 def logarithmic(y,n):
9     k = 1/(1+ln(4)) # special stiffness for this example
10    return k*[-(y-1)**2*ln(y),
11              -(y-1)*(y+2*y*ln(y)-1)/y,
12              1/(y**2)+2/y-2*ln(y)-3][n] if y < 1 else 0
13
14 # n'th derivative of filtered quadratic stiffened logarithmic energy [Huang et al. 2023]
15 def quad_stiffened_logarithmic_filtered(y,n):
16     k = 1/(3*(ln(4)+3)*ln(4)) # special stiffness for this example
17     thr = 19602/25+2277*ln(10)/5-586*ln(10)**2/25 # curvature cap evaluated at y=0.1
18     tmp = [(y**2-1)**2*ln(y**2)**2,
19            4*(y**2-1)*(y**2*ln(y**2)+y**2-1)*ln(y**2)/y,
20            8*y**2*ln(y**2)**2-4*(1-y**2)*ln(y**2)**2-32*(1-y**2)*ln(y**2)
21            -4*(1-y**2)**2*ln(y**2)/y**2+8*(1-y**2)**2/y**2]
22     tmp[2] = min(thr, tmp[2]+tmp[1]/(2*y)) # filter curvature
23     return k*tmp[n] if y < 1 else 0
24
25 # n'th derivative of total energy
26 def energy(y,n,barrier):
27     return barrier(y,n) + momentum(y,n)
28
29 # Newton's method
30 def solve(y,barrier):
31     j = 0
32     for k,_ in enumerate(iter(int,1)):
33         e,g,c = [energy(y,i,barrier) for i in range(3)]
34         d = -g / c
35         if d**2 > 1e-8:
36             alpha = 1
37             while y+alpha*d <= 1e-8 or energy(y+alpha*d,0,barrier) > e:
38                 alpha, j = alpha * 0.5, j + 1
39             y += alpha * d
40         else:
41             print( f'converged_to_{y:.3f}_at_{k}_steps_{j}_line_search' )
42             break
43
44 eval_y = 1e-3
45 solve(eval_y,logarithmic)
46 solve(eval_y,quad_stiffened_logarithmic_filtered)
47
48 # prints below:
49 # converged to 0.500 at 12 steps 0 line search
50 # converged to 0.500 at 5 steps 6 line search

```

Listing 1. Python Code for Convergence Analysis